

Reconstruction of encoded data in DNA storage technology

Subhasiny, Sankar

2022

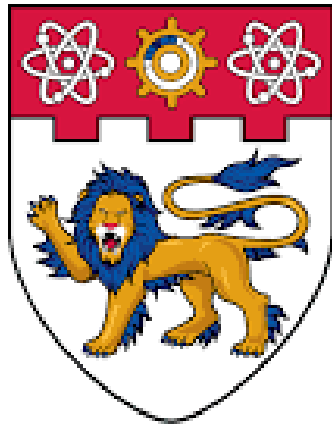
Subhasiny, S. (2022). Reconstruction of encoded data in DNA storage technology. Master's thesis, Nanyang Technological University, Singapore. <https://hdl.handle.net/10356/156818>

<https://hdl.handle.net/10356/156818>

<https://doi.org/10.32657/10356/156818>

This work is licensed under a Creative Commons Attribution-NonCommercial 4.0 International License (CC BY-NC 4.0).

Downloaded on 23 Aug 2022 11:02:06 SGT



**NANYANG
TECHNOLOGICAL
UNIVERSITY**

SINGAPORE

**RECONSTRUCTION OF ENCODED DATA IN DNA STORAGE
TECHNOLOGY**

SANKAR SUBHASINY

SCHOOL OF ELECTRICAL AND ELECTRONIC ENGINEERING

2022

RECONSTRUCTION OF ENCODED DATA IN DNA STORAGE TECHNOLOGY

SANKAR SUBHASINY

School of Electrical & Electronic Engineering

A thesis submitted to the Nanyang Technological University
in partial fulfilment of the requirement for the degree of
Master of Engineering

2022

Statement of Originality

I hereby certify that the work embodied in this thesis is the result of original research, is free of plagiarised materials, and has not been submitted for a higher degree to any other University or Institution.

NTU NTU NTU NTU NTU NTU NTU NTU
NTU NTU NTU NTU NTU NTU NTU NTU
NTU NTU NTU NTU NTU NTU NTU NTU

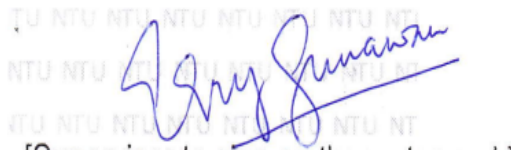
S. Subhasiny

24-01-22
Date

SANKAR SUBHASINY

Supervisor Declaration Statement

I have reviewed the content and presentation style of this thesis and declare it is free of plagiarism and of sufficient grammatical clarity to be examined. To the best of my knowledge, the research and writing are those of the candidate except as acknowledged in the Author Attribution Statement. I confirm that the investigations were conducted in accord with the ethics policies and integrity standards of Nanyang Technological University and that the research data are presented honestly and without prejudice.



24-01-22
Date

ERRY GUNAWAN

Authorship Attribution Statement

*(B) This thesis contains material from paper(s) submitted at conferences in which I am listed as an author.

Chapter 4 is submitted as Subhasiny, Wang Yixin, Zhang Jiayu, Nur Sabrina, Erry Gunawan, Yong L.Guan, Noor-A-Rahim, Chueh L. Poh "Comparative Analysis of clustering methodologies in DNA Data Storage", IEEE Symposiums on Computers and Communications, ISCC, 2022

The contributions of the co-authors are as follows:

Prof Erry Gunawan, Wang Yixin and Dr. Noor-A-Rahim provided the initial project direction.

I prepared the manuscript drafts, and it was edited by Wang Yixin. The manuscript was revised by Prof Yong L. Guan and Dr. Noor-A-Rahim.

I used the real time dataset provided by Wang Yixin which she obtained by performing study in lab associated with Prof Chueh L. Poh and Dr. Noor-A-Rahim.

All works such as designing, experimenting and validating the clustering methodologies was conducted by me.

Zhang Jiayu and Nur Sabrina designed and estimated parameter for 2 experiments

Wang Yixin and Prof Yong L. Guan assisted in analysing best clustering methodology.

Prof Chueh L. Poh, Dr. Noor-A-Rahim and Prof Yong L. Guan assisted in the interpretation of the dataset.

24-01-22
Date

SANKAR SUBHASINY

Abstract

Inevitable biomolecular errors in DNA storage technology could be resolved by designing robust error correction codes or intelligent clustering/decoding algorithms. The first objective of our work is to reconstruct the encoded DNA sequences read from the Illumina sequencer before decoding by studying the efficiencies of the existing clustering tools in the biological domain and then modifying, tuning, and analyzing their applicability in the DNA data storage domain. The investigated tools and algorithms include Starcode, Cooperative Sequence Clustering, Majority nucleotide selection algorithm, Slidesort, and MeShClust. We observed and compared them, Starcode, Majority nucleotide selection algorithm and Cooperative Sequence Clustering yields the highest recovery rate with less sequencing redundancy for three datasets. The benefit of portability using nanopore-based storage leads to the second objective of designing a Nanopore based DNA storage simulator that can serve as a tool for evaluating coding and clustering techniques. We simulated the DNA channel and subsampling of sequenced data using the non-parametric subsampling method by studying the distribution of real nanopore DNA storage data and then integrated it with DeepSimulator. The design is evaluated for its accuracy by comparing it with real nanopore reads. Besides, nanopore reads obtained from the designed simulator are clustered and representatives in each cluster are extracted for reconstructing the encoded data.

Acknowledgements

In this acknowledgment, I intend to thank all those who were involved in determining the shape of this report and helped me immensely so far directly or indirectly. I would like to take this opportunity to express my deep sense of gratitude and profound feelings of admiration to my Supervisor **Prof Erry Gunawan**, and Co-Supervisors **Prof Guan Yong Liang, Prof Noor-A-Rahim, and Prof Poh Chueh L.Poh**. I am indebted to **Wang Yixin** who has always kept her hands of blessings on my head and held my hands tightly in this world of research. I have been enriched with their advice and have implemented those innovative ideas in my M. Eng thesis report. I also thankfully acknowledge the assistance received from **Nur Sabrina Binte Iskandar Abdullah Yeoh and Zhang Jiayu**.

Table of Contents

Statement of Originality	i
Supervisor Declaration Statement	ii
Authorship Attribution Statement	iii
Abstract	iv
Acknowledgements	v
Table of Contents	vi
List of Figures	viii
List of Tables.....	x
Chapter 1 Introduction	1
1.1 Motivation and Objective.....	1
1.2 Contribution	3
1.3 Organization	4
Chapter 2 Background and Literature Review	5
2.1 DNA Data Storage Prototypes	7
2.2 Simulators depicting DNA Data Storage channel.....	9
2.3 Clustering tools in biological domain.	10
2.3.1 Illumina based clustering software tools.....	10
2.3.2 Nanopore based clustering tools	10
Chapter 3 Problem Claim and Methodology.....	12
3.1 DNA Data Storage System.....	12
3.2 Clustering	15
3.2.1 Starcode.....	16
3.2.2 Slidesort.....	17
3.2.3 MeShClust.....	17
3.2.4 Majority Nucleotide Selection Algorithm.....	18
3.2.5 Cooperative Sequence Clustering	19
3.2.6 isONclust.....	19
3.3 Material and Methods for Proposed Simulator	20
3.3.1 Difference of Simulator Requirement in Biological and DNA Storage domain.....	20
3.3.2 Components in DeepSimulator	21
3.3.2.1 Sequence generator	21
3.3.2.2 Signal generator (Pore Model).....	21
3.3.2.3 Base callers.....	23
Chapter 4 Comparative analysis of Illumina based Clustering tools	25

4.1	Post Processing in Slidesort	25
4.2	Modified GLM in MeShClust	26
4.3	Tuning and Comparative analysis of all algorithms.....	27
4.4	Summary	33
Chapter 5 Design of Computational Simulator for DNA storage (based on nanopore sequencing)		34
5.1	Detection of distribution.....	34
5.2	Statistical Analysis	36
5.3	Implementation of Copy Distribution for various Coverage values	40
5.4	Preprocessing module and Computing Confidence Interval and Coefficient of Variations for all distributions.....	41
5.5	Integration of Preprocessing module in DeepSimulator	43
5.6	Validation of the Proposed Simulator	44
5.7	Downstream processing of Simulated sequences.....	52
5.8	Summary	56
Chapter 6 Conclusion		57
6.1	Research Outcome.....	57
6.2	Recommendations for future research.....	58
PUBLICATIONS		59
REFERENCES.....		60

List of Figures

Figure 1: Growth and digitization of information storage capacity (Images source from wiki page of big data).....	5
Figure 2: Illustration of a general architecture of DNA storage	12
Figure 3: Oligo losses and errors.....	14
Figure 4: Clustering tools.....	15
Figure 5: NW algorithm	16
Figure 6: Penalty Matrix	17
Figure 7: Lookup Table.....	17
Figure 8: DNA storage architecture	18
Figure 9: Pictorial Representation of Deep Simulator	22
Figure 10: Proposed Post Processing in Slidesort.....	26
Figure 11: Comparison of recovery rates in MeShClust with alignment, non-alignment (GLM) and modified GLM.....	27
Figure 12: Comparison of recovery rates calculated from different tools	29
Figure 13: Comparison of decoding complexity, i.e., the number of clusters after applying clustering mechanism.....	31
Figure 14: Comparison of distribution between ground truth (base computed distribution) and after clustering.....	32
Figure 15: Bootstrap Sampling	36
Figure 16: Statistical analysis for evaluating sampling methodology.....	37
Figure 17: Histograms of sequences and distribution	38
Figure 18: Interval of mean. Comparison of original mean with simulated mean in different sampling methods. Observations: A - For Maximum iteration, the observed mean is equal to original mean, B - For Maximum iteration, the observed mean is greater than original mean.....	40
Figure 19: Histograms for different coverages	41
Figure 20: Flowchart for integrating sequencing coverage and synthesis error	42
Figure 21: Computing Confidence Interval	43
Figure 22: DNA Storage Channel Simulator- Dotted rectangle is integrated in existing DeepSimulator for simulating steps of synthesizing, PCR, sequencing certain amount of data (determined by specific coverage).....	44
Figure 23: DWT warped signal (a_1 warped signal – simulated signal and a_2 warped signal – Real Nanopore signal)	46
Figure 24: Analysis of signals: Simulated signals from the proposed simulator having the least sample difference (with respect to 2 real nanopore signals) are taken and represented by pictorial representation after performing DWT and cropping.	49
Figure 25: Biomolecular Errors in nanopore Sequencer (O1), proposed Simulator (S1-S11) and DeepSimulator (D1)	50
Figure 26: Biomolecular Errors with primer (M1-M4), without primer (N1-N4)	51
Figure 27: Flow diagram- Clustering, Clustering metric calculation and Evaluation of tool.....	52
Figure 28: Possibility to recover data.....	54
Figure 29: Formation of clusters with count	54
Figure 30: Completeness.....	54
Figure 31: Homogeneity	54
Figure 32: Error rate vs Accuracy	55
Figure 33: Comparison of varied error rate for all sequence and representative sequence	55

Figure 34: Sequence length distribution.....	56
--	----

List of Tables

Table 1: Parameter Tuning	28
Table 2: Recovery rate by applying clustering algorithm on 3 datasets	29
Table 3: Distribution of all experiments	35
Table 4: Student t-test statistical result	37
Table 5: Confidence interval tuning using standard error method.....	39
Table 6: Confidence Interval and Coefficient of Variation for various distribution.....	43
Table 7: Sample difference calculation between nanopore signal and simulated signals (having sample length of around 4000 due to biomolecular error after DTW) mapped to input encoded sequence of length 160	45
Table 8: Sample difference calculation between two nanopore signals and simulated signals (having sample length of around 4500 due to biomolecular error after DTW) mapped to input encoded sequence of length 160.....	47
Table 9: Observation 1- Sample difference calculation of two nanopore signals of the same original sequence and simulated signals (having sample length of around 4000 due to biomolecular error after DTW) mapped to input encoded sequence of length 160	48
Table 10: Observation 2 - Sample difference calculation of two nanopore signals of the same original sequence and simulated signals (having sample length of around 3000 due to biomolecular error after DTW) mapped to input encoded sequence of length 160	48

Chapter 1 Introduction

1.1 Motivation and Objective

Higher density and longevity make deoxyribonucleic acid (DNA) a promising medium for archival. For extraction of stored data from DNA molecules, sequencing/reading is performed where a large pool is amplified, and mutated data is subsampled. Various clustering techniques to cluster the sequenced reads (of Nanopore and Illumina sequencing) have been proposed in the Genomic as well as DNA storage domain. In our study, we discuss the coverage size (denoting the number of times each base must be read) required based on different clustering techniques to reconstruct the encoded DNA sequences before decoding. Motivated to reduce the decoder complexity, this research aims to explore the features and limitations of the clustering techniques and to find out the best clustering technique which could take a minimum number of sequenced reads also termed as sequencing redundancy, and recover the encoded sequences with maximum similarity for Illumina Sequences. Owing to its portability, the nanopore sequencer is best suited for storing data than Illumina but it has higher error rate. Hence there is a need to conduct enormous number of real-time trial experiments to determine the best encoding/decoding methods and downstream processing. And good basecalling accuracy must be selected for finding out optimum coverage during clustering. To reduce the considerable number of real-time trial experiments that could aid in finding out the encoding/decoding and clustering techniques, we got inspired by the idea of creating a Computational Simulator tool. We design a simulator incorporating certain pre-processing mechanisms in the existing nanopore model created for biological application for simulating nanopore sequences/signals and validating the simulated result with original nanopore result. Besides, the work will also explore the approaches to cluster the simulated data with varied basecalling accuracy and determine the minimum coverage for reconstructing the encoded sequences. This study tells us how best simulator can be utilized by moderating its tunable parameters thereby making similar set up in real-time.

The specific aims are

- To experiment with various clustering techniques and find out the better method to cluster sequences read from Illumina. The better method is evaluated based on the minimal coverage size and maximal recovery rate.
- To design a computational simulator model for DNA storage domain consisting of the process of Synthesizing, PCR, and Sequencing (Write, Store and Read)
- To analyze the biological clustering mechanism, determine the basecalling (Intermediary process: conversion of nanopore signals to basecalled sequence) accuracy and the best minimal coverage size required to cluster sequence with high recovery rate. The error rate

of all sequenced data is compared with the error rate of representative data for validating clustering methods along with other assessment metrics such as homogeneity, completeness, error rate and sequence length distribution.

1.2 Contribution

The specific contribution of this thesis are as follows

- Modification of the clustering tools such as Slidesort, Starcode and MeShClust designed for biological domain to fit in DNA storage domain. Comparison of these three tools with Majority nucleotide selection algorithm and Cooperative sequence clustering. It is observed in one dataset for coverage of 8.6x (x represents number of times one synthesized sequence must be read from sequencer), Starcode outperforms with an 89.89% of recovery rate, almost 30% higher than Slidesort and MeShClust and 2% higher than the Majority nucleotide selection algorithm. Additionally, it is also found that after processing with Majority nucleotide selection, the number of sequences sent to the decoder is reduced the most, thereby reducing decoding complexity. To generalize the best clustering technique for all DNA storage data, another two datasets are also clustered with various techniques, Starcode and Cooperative Sequence clustering outperforms compared to other clustering tools in terms of higher recovery rate with least coverage size. Other evaluation metrics such as decoding complexity, speed and distribution is also taken into consideration.
- Designed a computational Nanopore-based DNA data storage simulator to approximate data distribution and data errors at the receiver of the actual experiments. The derived distribution is integrated with the DeepSimulator, which simulates the real-time nanopore sequencing. The sample differences computed between real nanopore signals and simulated nanopore signals inform that the simulated signals resemble closer to the actual nanopore signal. Specifically, we observed that signals from the proposed simulator have 16.7% to 88.7% lower sample difference deviation than that of signal simulated from DeepSimulator. The generated sequences or raw signals from the proposed simulator provides a cost-efficient and time-efficient tool for assessing *physical and logical* redundancy for data reconstruction in DNA data storage. In addition, conducted a study on utilizing the computational simulator to generate reads and cluster it with the biological clustering tool ‘isONclust’ and determined the various coverage rate required for varied basecalling accuracy to achieve a higher similarity rate with original encoded sequence. For basecalling accuracy of above 86%, coverage of up to 30x is required and for basecalling accuracy of above 80% and below 86%, the similarity rate is very less even with higher coverage of 62x. Additionally, it is inferred that the representative sequence of each cluster has less Insertion, deletion, and substitution error compared to all sequences subsampled.

1.3 Organization

The organization of the report is as follows. Chapter 2 introduces the brief background of DNA data storage and a literature review on the existing DNA data storage prototypes, the advantage of one clustering method over others, and simulators simulating sequencing action. Chapter 3 discusses the simulators used in the biological domain and describes several clustering methodologies that can be used for clustering sequences. Chapter 4 discusses the modification of clustering tools and experiments with real-time Illumina sequences. Chapter 5 describes the modified variant of the simulator, validating the designed simulator and presents the research results of the clustering tool. Chapter 6 concludes this report and discusses the future work.

Chapter 2 Background and Literature Review

The medium used for storing data has varied across time with the advancement of technology from bones, rocks, and paper to CD, DVDs, and flash drives in recent days [1]. These storage media provide the storage density of GB range per inch square which is not sufficient today for storing large data of zettabyte range due to emerging technological development in all applications [2]. In this situation, cloud computing serves as a saviour of storage [3]. But need lies in finding alternative storage solutions. Moreover, storing the data for a prolonged time is impossible in the case of optical or magnetic material as it decays while preserving. To overcome these challenges of preserving data and storing a large amount of data, a new storage technique has to be designed. The four nucleotides of DNA [4] such as Adenine, Cytosine, Thymine, and Guanine provide twice the capacity of the binary system represented by '0' and '1'. Data stored in DNA can last for 2000 years at 9.4°C or 2 million years at -18°C and the expense is reduced thousand times compared to devices in traditional data centres [5]. And in a single gram of DNA, 215 petabytes of data can be stored. Using these two advantages, researchers are attracted to design an architecture required for DNA data storage. DNA storage technology has the potential to serve as a cost-effective solution for the digital archiving problem [6][7].

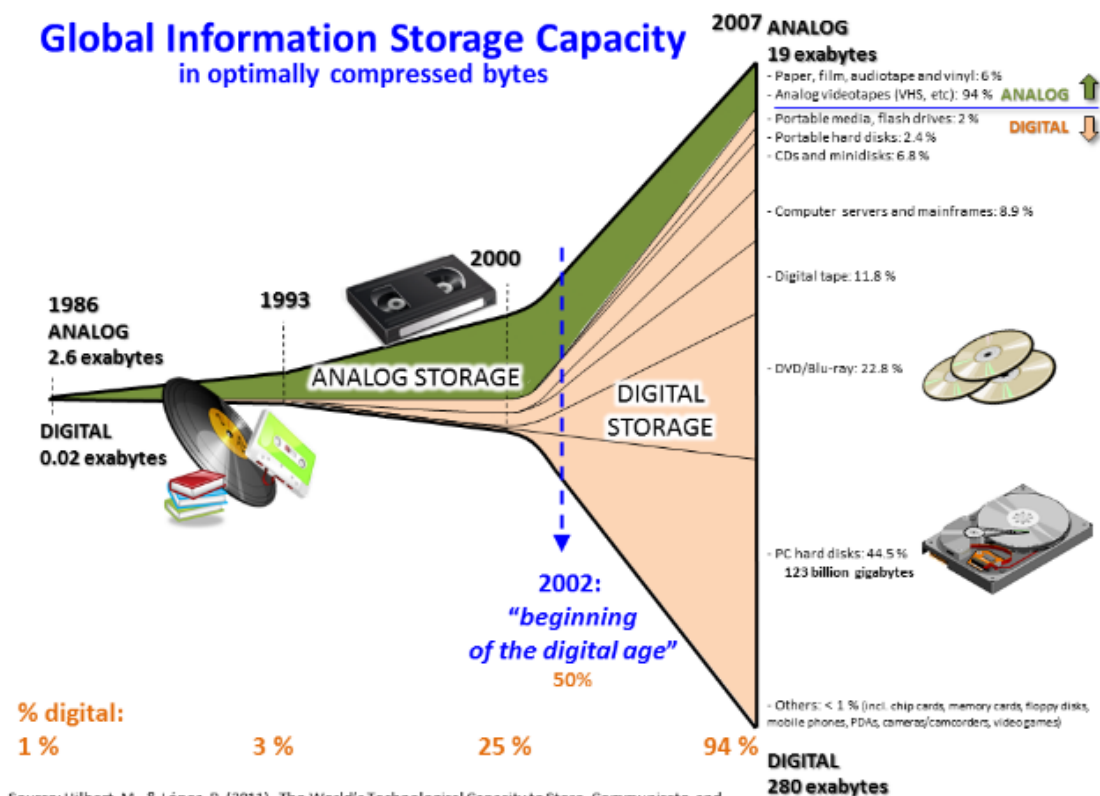


Figure 1: Growth and digitization of information storage capacity (Images source from wiki page of big data)

Similar to other storage devices, content inside the file is encoded. For storing data, it must be mapped to four nucleotides and then synthesized into biomolecular DNA in form of an oligo pool. The pool refers to a solution in which synthesized oligos are separately and stochastically stored. The pool is amplified with polymerase chain reaction (PCR) and then read using the sequencer [8]. During the process of DNA storage, each step will contribute to oligo losses and errors. There are two ways to recover the data despite the distortion with its *physical and logical* redundancy [5]. Reads may be lost while sequencing from certain subsamples. Hence, it is better to provide more coverage (which means number of readout sequences from sequencer out of total number of input encoded sequence) to retrieve all reads. But practically setting up high coverage increases the number of reads to sequence which in turn increases the complexity. Hence, there is a trade-off. To overcome this trade-off, the encoder and decoder must be designed appropriately with appropriate logical redundancy so that they can recover complete data from sequenced subsamples. Many codes have been devised to deal with erasure errors by increasing redundancy that occurs while storing data in all digital storage systems. The outer code adds redundancy in the form of additional sequences to recover lost sequences, and the inner code adds redundancy within each sequence, to correct errors on the sequence level (for example substitution errors). However, in the case of DNA storage systems implementing error-free architecture is much challenging. The sequenced output consists of oligo errors (insertion+, deletion-, substitution*). Hence clustering and then aligning by estimating representative sequence helps to find out the estimated input sequence thereby reducing decoder complexity. Here, also there is a trade-off with coverage. If we increase coverage, we can cluster sequences appropriately and estimate the original sequence as multiple sequences will be clustered. But the best clustering mechanism lies with less coverage and estimating exact sequence in-order to reduce the complexity of sequencer in addition to reduction in decoder complexity. The real-time sequencing can be controlled by determining the sufficient coverage required for successful decoding thereby saving time and number of runs.

The most preferably used sequencers (biological hardware device) are Illumina and Nanopore which have varied error rates. There are various sources of errors in nanopore sequencer due to inhomogeneous translocation speed, low signal-to-noise ratio (SNR), and multiple nucleotides passing through nanopore simultaneously. Uplift of current happening in nanopore sequencer leads to translocation which in turn leads to lower Phred quality score. Low SNR occurs because of structural similarity of nucleotides, non-uniform speed, and the fact that signal does not change within homopolymers. Hence the disadvantage of DNA Storage technology is the influence of biomolecular errors because of DNA's mutation. To design error control codes and suitable clustering techniques, the nature of each step such as Synthesizing, Storing, and Sequencing in terms of erasure and errors should be analyzed. For analyzing, Simulators can be used rather than real-time experiments. Many simulators are available simulating nanopore. Deep simulator mimics the nanopore sequencing process in the biological domain with oligo errors.

There are various clustering tools specific to Illumina and Nanopore sequencers. These clustering tools use a basic alignment algorithm with some advancement embedded through modification and then designing an algorithm that can process and cluster the data in less time and complexity. Optimum coverage size can be decided with a good clustering tool. Compared with a digital storage system, the other limitation of DNA data storage is the higher cost for performing each step. However, rapid advancement in DNA synthesis and sequencing technology assures decreased cost of using DNA as a storage medium. In the following sections, state of art DNA data storage prototypes are reviewed in Section 2.1. In Section 2.2, existing simulators for generating signals like biological nanopore signals are reviewed. And in Section 2.3, the existing clustering tool and its accuracy in clustering genomic sequences generated from Illumina and nanopore sequencer are described.

2.1 DNA Data Storage Prototypes

Several research groups predicted the sequencing coverage required for Illumina-based or Nanopore-based DNA data storage through real-time experiments. Illumina sequencing technique [9] having the advantage of lower cost and lower error rate (1%) was mostly used before Nanopore's discovery [10]. With Illumina as a sequencing platform, research groups predicted different sequencing redundancy for different encoded data sizes, coding, data reconstruction techniques, and logical redundancy. The sequencing coverage, clustering methods, and coding techniques used by different research teams are discussed. One of the most prominent DNA data storage architectures was framed by Church et al who successfully stored 0.65 MB of data in synthetic DNA with a physical density of 1.28PB/gram. The errors in synthesis and sequencing rarely appear in coincidence, hence by extracting multiple copies, it can be corrected. This approach required 3000-fold nucleotide coverage [11]. Goldman et al added data compression in a new DNA error control coding scheme and encoded 0.75 MB of data into DNA. They have modeled the influence of reduced sequencing coverage on the per-decoded-base error rate creating 4-fold redundancy by finding the optimum limit [12]. These two techniques require manual intervention for the correction and detection of errors. But through Reed Solomon (RS) coding by Grass et al, all the data can be recovered [13]. Since data is not randomly accessed, the drawback of reading 4 consecutive reads for rewriting a single base is complicated [14]. To address this problem along with random access of data, Yazdi et al developed rewritable DNA storage architecture with constrained coding[15].

A key aspect of DNA storage is to devise appropriate encoding schemes that can tolerate errors by adding redundancy. Redundancy is added in encoding like RAID5 through the XOR method by Bornholt et al. In this approach, tunable logical redundancy is implemented based on the criticality of each data set/point. In terms of sequencing coverage, they have used 0.1% of the total reads sampled and attained 50% accuracy and with 10% of total reads, they achieved 100% accuracy. Encoding becomes more resilient to errors at high sequencing depth [16]. Recently, DNA data storage based on fountain code was proposed by Erlich et al where in the deep copy extraction, sampling sub-sequences

implies highly skewed negative binomial distribution. They conducted down-sampling experiments with different coverages and found that with a size parameter of $r=7$ (in negative binomial distribution) and with a coverage size of 7, a low drop-out rate of 0.5% can be achieved [17]. To improve the decoding performance of fountain code, clustering is performed based on hamming distance. Q-score sorting and discarding strategy are followed to provide better results [18].

Organick et al [19] reduced the sequencing read coverage required for error-free decoding by maximizing information in all sequenced reads. They encoded 200MB of data and by maintaining logical redundancy of 15% added through RS code similar to prior works, they could recover data with low sequencing coverage provided Illumina sequencer used. They performed clustering through similarity identification by locally sensitive hashing and lightweight checks based on binary embedding [20]. Then the clusters are processed to recover the original sequence through modified Bitwise Majority Alignment algorithm (BMA) forming a matrix with errors and erasure. This is then corrected through the decoder. Error resilience is achieved in the median coverage of 6.2. And the distribution observed is approximated as gamma.

Using Repeat Accumulate code and mapping of variable length nucleotides based on Illumina storage, Wang et al recovered the erroneous data with minimum coverage of 10x having a considerably small drop-out rate of 3.3% [21]. With a single large block length Low-Density Parity-Check code, the read/write trade-off was improved by Chandak. et.al for Illumina-based DNA storage, and it was inferred that coverage variance plays a key role in decoding performance, especially for codes with lower redundancy [22].

Recently, Nanopore sequencing [23][24] has been attracting attention due to its portability albeit with a higher error rate of 15%. Fortunately, setting up higher sequencing redundancy can help in dealing with these high error rates. The clustering and decoding approach which was implemented in Illumina is tested for Nanopore reads and it is observed that even when coverage is tuned between 74x to 147x, recovering the file seems to be impossible. Hence, they suggest using large number of reads during the process of clustering and decoding [19]. High error rates and limited sequencing throughput remain significant limitations for this technology [25]. Another experiment also observes that in nanopore sequencer, more reads are required to increase its recovery rate compared to Illumina [26]. To increase the DNA fragment length, a new workflow consisting of Gibson Assembly and Overlap-Extension polymerase chain reaction (OE-PCR) was implemented iteratively and for decoding, an earlier consensus algorithm in [19] is modified thereby successfully decoding with lower coverage of 22x. In a work, clustering was performed with several multiple sequence alignment techniques such as Kalign [27], clustal omega [28], muscle [29], coffee, and aggregate consensus was computed based on the majority of homopolymers. With this, error reduced to 0.02% and the sequencing coverage depth varies for each sequence between 75x to 566x [30].

The raw signal obtained from the nanopore sequencer has more information than the basecalled reads but the raw signal is difficult to decode. Hence Chandak et al have proposed an algorithm for performing decoding on intermediate transition probability output from Flappie basecaller working on recurrent neural network. They have observed a 3x improvement in reading cost at the same writing cost. They generated a read/write trade-off similar to Illumina-based DNA storage in their proposed algorithm [31].

To summarize, researchers have conducted independent trial experiments to explore the minimum *physical and logical* redundancy (in code design) required for data recovery in DNA data storage with different system settings, costing highly in time and money. Instead, such exploration and validation could be done through a simulator, saving time and several runs.

2.2 Simulators depicting DNA Data Storage channel

We were inspired by the work of Chen et al to develop a model for DNA data storage using the third-generation sequencing device to benchmark new methods for encoding/decoding and data reconstruction techniques. They developed a statistical model of DNA data storage provided that the sequencer is Illumina and represented the whole molecular process by a statistical model, i.e., synthesizer through normal distribution, PCR by stochastic branching process, and sequencing by random sampling [32]. This model helps to analyze and decide the redundancy required for the experiment rather than performing lots of trial and error. The computational model provides a useful tool for tracing the performance of DNA data storage with different settings.

Result from [33] show that insertion and deletion rates are quite higher in synthesis and substitution error rate dominates in the sequencing phase. Sequence loss happens due to coverage distribution. Another study informs that low redundancy set up at synthesis phase leads to variation in distribution from negative binomial to long-tail with a peak at 0 and leads to PCR bias as PCR cycles are required to be increased. This ends up in the erasure of sequences [34][35].

Deep Simulator [36] is a model (computational software) which mimics the real nanopore sequencer by providing the result in the form of a signal and then basecalling it with standard basecallers for obtaining nucleotide sequences. There are two versions developed. In the first version, the priority was given to Context dependent pore model and albacore basecaller was integrated. In the second version, context independent pore model was given a priority. In addition, to simulate the reads similar to real nanopore reads, low pass filter was added and integrated with recent official basecaller called Guppy supporting both GPU and CPU. Instead of repeat time distribution in DeepSimulator 1.0, Deep Simulator 1.5 [37] adds repeat time in simulated signal based on pattern in real signal through deep learning model. In addition, DeepSimulator 1.5 and 1.0 are compared with real nanopore signal through quantitative analysis of continuous wavelet Transform (CWT) spectrum.

2.3 Clustering tools in biological domain.

2.3.1 Illumina based clustering software tools

Slidesort is found to be better than naïve matching, neighbor pair and normal k-mer hash approach [38]. Starcode is compared with Slidesort and though Slidesort is an exact matching algorithm, it is observed to produce 5-10% fewer 3 matches than Starcode on all biological datasets [39]. MeShClust is compared with other clustering algorithms through its evaluation criteria such as intracluster similarity, inter-cluster similarity, silhouette score, purity, and Normalized Mutual Information (NMI). MeShClust [40] stood top and CD-HIT[41] follows it whereas DNACLUSt[42] does not obtain a good score. MeShClust is the first clustering algorithm in bioinformatics to give an option with machine learning and alignment approach for detecting sequence similarity. Another tool called SpCLUSt is also designed with machine learning. The advantage of SpCLUSt compared to MeShClust is that there is no need to specify any sequence identity parameter or centroid sequences user input or similarity threshold (when it is hard to specify in certain applications) [43].

2.3.2 Nanopore based clustering software tools

In the biological domain, certain clustering tools have been developed for clustering long nanopore reads which fall under two categories such as graph-based and community-based. Graph-based clustering works by searching minimum cuts in graphs and estimating the number of clusters whereas community-based clustering works on the strategy of finding dense subgraphs by measuring link density. Several types of clustering methods such as Modularity [44], Clique percolation [45], Louvein [46], CARNAC-LR [47], and isONclust are followed for clustering biological data sequenced from nanopore sequencers. Specifically, the first step is to group reads, and then the second step is to estimate the representative by correcting the errors using the information within each cluster.

Most of these methods work with a community-based approach or combination of both community-based and graph-based clustering methods. For any clustering algorithm, an important consideration should be given to accuracy and speed and whether it can deal with the complexity of sequences such as large length, a large number of sequences, and evolutionary distance.

Modularity, Clique percolation, and Louvein suffer from drawbacks such as dealing with small clusters and clustering orders. This is rectified by Clustering Coefficient Based Acquisition of RNA Communities in Long Reads (CARNAC-LR) which is robust to noise. In addition, however clustering tools: Modularity, Clique percolation, and Louvein highly depends on the high accuracy provided by Pacific Biosciences Consensus Circular Sequence (CCS) reads, and therefore do not apply to ONT reads [47]. In CARNAC-LR, through minimap2 [48] which works faster than Burrows-Wheeler transform

[49], the similarity is found, and it is followed by clustering based on threshold through a graph. Recall, Precision, F-Measure, and Jaccard index tell that CARNAC-LR is better than other approaches [50].

But CARNAC-LR sub-optimally performed while experimenting with large datasets. With quality value, the clustering accuracy can be improved further especially for a higher error rate [51] in Nanopore sequences similar to [18] in the case of Illumina sequences. isONclust uses quality value and minimizer approach for clustering [52]. Moreover, isONclust avoids all to all similarity comparison and hence it is suitable for highly scalable applications. Many reasons help us to conclude that isONclust has superior performance compared to CARNAC-LR. isONclust is fast and consumes only 1630 minutes compared to CARNAC-LR which consumes 5053 minutes. Memory usage is less in isONclust. And isONclust provides good clustering metric accuracy such as Homogeneity, Completeness, V-measure, and ARI. CARNAC-LR can only deal with nanopore reads having a 7% error rate whereas isONclust can deal with reads having an error rate of up to 20%. Hence with these advantages, we have chosen isONclust for clustering simulated reads based on nanopore sequencer. As nanopore sequences are more prone to error, we need to use quality value and isONclust uses quality value in its algorithm thereby reducing the error from the clustered dataset as experimented in Section 5.7, Figure 32.

Chapter 3 Problem Claim and Methodology

In this chapter, we introduce the preliminary background of DNA data storage as well as alignment techniques leading to the building up of clustering tools. In Section 3.1, the DNA as a storage medium and its disadvantage in terms of errors are described in detail. In Section 3.2, the clustering techniques for Illumina and Nanopore sequences are discussed. In Section 3.3, a biological nanopore simulators are described.

3.1 DNA Data Storage System

In the DNA storage framework, as shown in Figure 2, the first step is encoding where the binary data is encoded with error control codes and then it is mapped to four nucleotides such as Adenine, Guanine, Cytosine, and Thymine. Secondly, the encoded DNA sequences are synthesized/written into biomolecular DNA termed as oligos. These DNA molecules are in the form of an oligo pool and stored under favourable conditions. Following that, amplification of molecules happens through the PCR process and Sequencing/reading is performed by randomly selecting the subset of data from the pool. Thirdly, the sequenced read obtained is clustered and representative sequence is estimated so that number of sequences to process in decoder will be reduced, and as a result of clustering, the encoded DNA sequences can be reconstructed. Finally, the clustered representative reads are decoded and mapped back to binary data. A higher recovery rate lies in the success of encoding/decoding technique and sequencing operation which depends on sequencing coverage as well as error in the model.

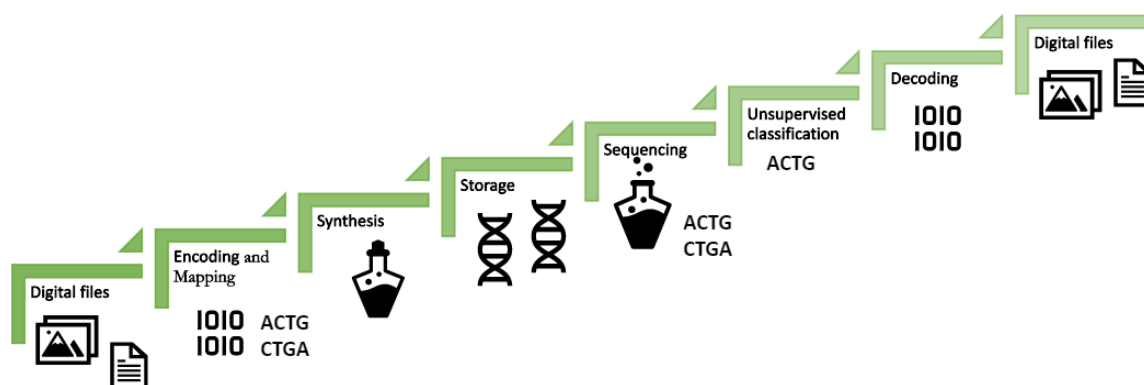


Figure 2: Illustration of a general architecture of DNA storage

In these steps of writing, storing, and retrieving the data, the sequence might get lost, and also it might get erroneous because of biomolecular errors such as insertion, deletion, and substitution embedded during the steps of synthesizer, PCR, and Sequencer. The error model completely depends on the

different biochemical processes. There are different synthesis processes such as solid-phase phosphoramidite chemistry, array-based DNA synthesis (using a series of electrochemical techniques on microarray chips), and enzymatic synthesis. Among these synthesis methods, array-based synthesis is highly preferred because of its speed, efficiency, and cost-effectiveness [2]. The popular DNA sequencing methods are Illumina and Nanopore. Illumina is a second-generation device that works based on image processing and a concept called sequencing by synthesis. Here, the individual base sequence is determined by spatial fluorescence pattern. Its benefit is lower cost and lesser error of 1%. Real-time streaming of sequenced data is performed using a long-length DNA sequencing technique called Nanopore, a third-generation device consisting of several nanopores controlled by Application Specific Integrated Circuits. It works by detecting the ionic current while passing the nucleotide molecule through the nanopore. Later the raw current signal is converted into nucleotides called FASTQ file using base caller. Basecaller such as Guppy/Flappie is developed with the neural network model (by training the model with real dataset) which identifies signal output and maps to sequence output. One important parameter in the basecalled FASTQ sequenced file is Q-score. With Q-score $Q(k)$, the probability (P) of how much we can trust the base-calling system for each base during sequencing can be determined as shown in below Equation 1.

$$P = \prod_{k=1}^{152} \left(1 - 10^{-\frac{Q(k)}{10}} \right) \text{----- Equation 1}$$

Though Nanopore is portable than Illumina, it has a higher error rate of 15% which can be dealt with by setting up higher physical redundancy through coverage thereby recovering the data. To combat the biological mutation that happens during the various steps of DNA storage as shown in Figure 3, assessment of *logical and physical redundancy* serves as a major need. Logical redundancy is added while encoding data into DNA sequences. Logical redundancy is embedded through various coding techniques such as Linear block code, cyclic code, BCH code, Repeat accumulate code, Reed-Solomon code, etc. Biochemical constraints such as GC content (Percentage of Guanine and Cytosine in DNA molecules) of around 50% and homopolymer runs of 3nt are considered while designing error control code to prevent mutation [53]. Physical redundancy is the physical copy of a given DNA sequence. This can be decided by analysing the error model and efficiency of various clustering tools depending on the sequencing technique used. Stochastic and systematic errors [54] are added up after sequencing through Nanopore sequencer while basecalling because of steric interference (caused because of chemical reactions), electrical noise, read length, certain K-Mer call (nucleotide of length 4-mer means K=4), GC content, positional specific leading to major source of errors such as inter symbol interference, and skipping of certain period while basecalling a signal. These errors can be reduced by designing a model with optimum *logical and physical redundancy*.

A higher recovery rate in DNA data storage irrespective of biomolecular errors is achieved by designing robust Encoding/Decoding and Clustering techniques with sufficient sequencing redundancy (comparable to physical redundancy). Here, sequencing redundancy is equivalent to coverage which means the number of readout sequences from the sequencer out of a total number of input encoded sequences. Practically, setting up high coverage increases the number of reads to sequence thereby increasing the recovery rate. However, this in turn also increases the cost and the complexity dealt with by the sequencer and decoder. Hence there is a trade-off between recovery rate and sequencing complexity. Decoding complexity relates to the size of data input to the decoder for data recovery. If there is no pre-processing before decoding, decoding complexity equals sequencing complexity. To reduce decoding complexity, pre-processing/clustering comes into play by passing only the estimated representative sequence to the decoder. To reduce sequencing complexity, an optimal clustering mechanism clusters data with minimum coverage and the estimate of the representative sequence resembling the encoded DNA sequences must be identified. Clustering tools for clustering the sequenced read emerges because of merging up various alignment algorithms with some modification in the process.

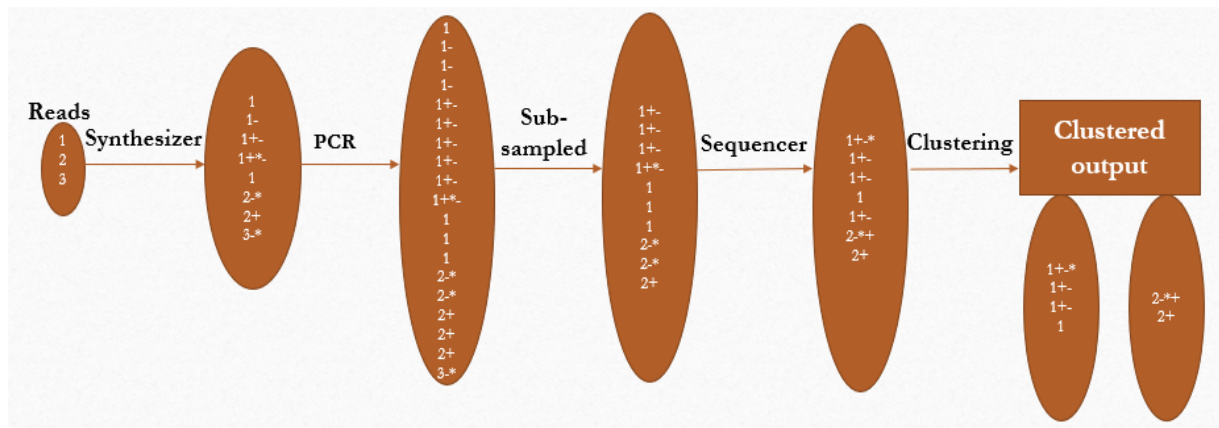


Figure 3: Oligo losses and errors

As seen in Figure 3, let us say we pass 3 reads to synthesizer, it gets amplified and either it remains the same or randomly affected by insertion (+), deletion (-) and substitution (*) errors or randomly affected by the combination of errors (+-, +*, -*, +-*). The same effect is observed in PCR stage and by setting up necessary coverage, sub sampled sequence is being sequenced by sequencer and certain losses are observed in that stage. In this example it is 3 that got lost completely. Clustering is performed to cluster and estimate representative sequence which is then passed to the decoder.

3.2 Clustering

There are various Clustering tools available for clustering the DNA sequences. And these clustering tools combine the alignment techniques along with certain modifications or implementation of additional algorithms. Alignment techniques are categorized based on either exact matching or approximate matching. Exact matching algorithms are Naïve matching, Boyer Moore algorithm, and multimap data structure such as Index and Hash. The exact matching doesn't suit practically since there is a variation due to biomolecular errors leading to the dissimilarity of two strings that can be measured by Edit/ Levenshtein and Hamming distance. Edit distance is defined as the minimum number of substitutions, insertion, or deletion that requires turning one string into another. Hamming distance is equal to the number of substitutions required to turn one string into another. Dynamic programming [55] algorithm helps to look for approximate occurrences of pattern with insertion, substitution, and deletion. This will also allow us to calculate the string similarity with penalties and to solve many bio sequence analysis problems including global and local alignment. Some of the approximate matching algorithms are Needleman-Wunch (NW) and Smith-Waterman (SW) algorithm and it works based on the edit distance matching problem. Depending on the sequence length and error rate, for Illumina Sequencing technique, the clustering tools that can be used are Starcode, Slidesort, MeShClust, etc (Figure 4).

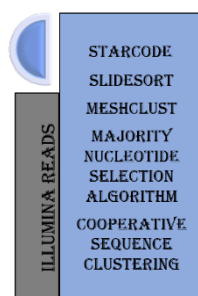


Figure 4: Clustering tools

The clustering tools for Illumina sequences [56][57] can be categorized by models used for clustering. They are connectivity, centroid, distribution, density, and neural. Connectivity-based clustering creates a hierarchical decomposition of the set of data using some criteria. Slidesort falls under this model. Centroid-based clustering constructs different partitions and evaluates them by some criterion. Starcode falls under this model with message passing where clusters are built by the bottom-up approach. Density-based connectivity is based on density function and MeShClust falls under this model. Distribution based connectivity model uses an algorithm such as Expectation maximization that tries to infer latent variables through an iterative approach for finding out maximum likelihood estimates of statistical parameters. Statistical values can be mean, variance, internucleotide covariance quartiles, median, confidence intervals, asymptotic properties, etc [58].

3.2.1 Starcode

Starcode [39] is based on all-pairs search, i.e., all pairs of the sequence below a given Levenshtein distance are identified during the graph construction phase. The graph is constructed using the NW algorithm. The edit distance finding problem aids in the approximate global alignment (NW algorithm) between two sequences by tracing the corresponding path to locate the error at each position as given in Figure 5.

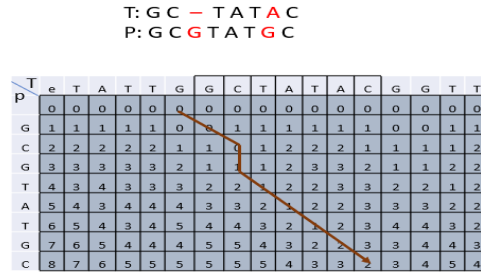


Figure 5: NW algorithm

The minimum edit distance in the last row is 2. The path is traced by finding out the value yielding 2 i.e., value can be a diagonal or adjacent element. When we track completely from back and reach up, the sequence following path is determined which helps in predicting the indels and substitution errors. The penalty for the substitution between purines (A and G) and pyrimidines (C and T), insertion, and deletion can be changed using penalty matrix which will help in analysing the nature of mutations. NW algorithm is modified in Starcode with the poucet strategy, which uses the redundancy of alphabetically sorted sequences to avoid unnecessary recomputations and to gain speed thereby eliminating the need to compute the entire matrix. Matching is carried out by lossless filtration, followed by an exhaustive search on the branches of a prefix tie. It can process data with varying sequence lengths. The alignment algorithmic Equation 2 is as follows

$$align(\alpha x, \beta y) = \min\{(align(\alpha, \beta) + p(x, y)), (align(\alpha x, \beta) + p(x, -)), (align(\alpha, \beta x) + p(-, y))\}$$

----- Equation 2

Where Penalty matrix $p(x, y)$ is

	A	C	G	T	
A	0	4	2	4	8
C	4	0	4	2	8
G	2	4	0	4	8
T	4	2	4	0	8
	8	8	8	8	

Figure 6: Penalty Matrix

In the penalty matrix (Figure 6), number 2 represents the transition between purines or between pyrimidines. Number 4 represents transversions from purines to pyrimidines or vice versa and number 8 denotes the gap.

3.2.2 Slidesort

Slidesort [38] does all pair searches with an exact matching algorithm similar to the Indexing approach of forming a look-up table like Figure 7 (with k-mer and corresponding positional value/offset value) thereby using that table to align the sequence. In this tool, radix sort is done at pre-processing stage followed by pairwise comparison through common k-mers. The minimum spanning tree algorithm [59] is used for forming a growth pattern to cluster. There is a constraint that sequence length for comparing should be of the same length.



Figure 7: Lookup Table

3.2.3 MeShClust

MeShClust [40] guarantees to converge into a local optimal point which is the center of the cluster. Two major processes followed in this tool are classifying and clustering. Firstly, sorting is done based on sequence length with the shortest sequence at the top and the first sequence is taken as the query

sequence. Secondly, the query sequence is compared with all sequences using global alignment and classified as similar and dissimilar sequences based on the edit distance threshold set by the user. Thirdly features such as sequence length difference, Czekanowski similarity, Manhattan distance, Pearson coefficient, and Kulczynski distance are extracted from those sequences after converting them into K-mer histograms and GLM is trained with these feature sets. In the fourth step, the mean shift algorithm is applied. In this algorithm, similar sequences are grouped as a cluster and the mean of sequences in the cluster is the synthetic center, the sequence situated near to that is the new center/new query sequence. In the fifth step, the estimated query sequence is aligned with the rest of the sequence (not in the created cluster) for classifying or using GLM, classification is done. Again, the mean shift algorithm for clustering is repeated. The fourth and fifth step is repeated until all data gets clustered. The choice of classifying the similarities and dissimilarity using GLM or alignment is based on identity score. If identity score > 60%, GLM is used, and if identity score < 60%, global alignment is preferred. This tool is suitable for clustering long reads as well as sequences of varied length.

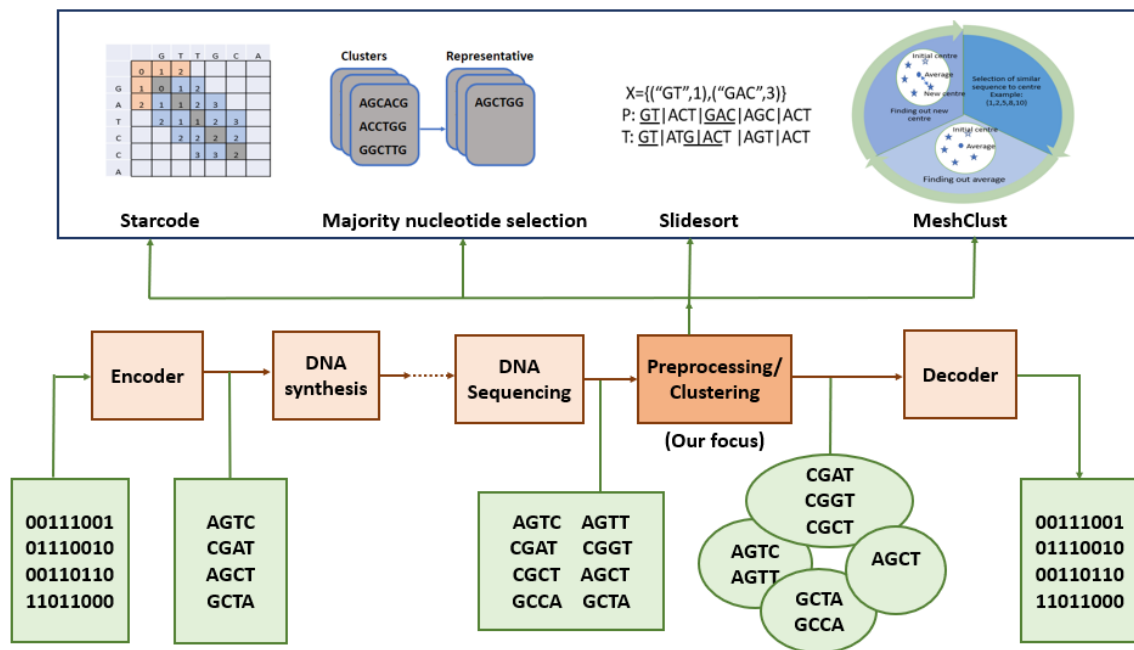


Figure 8: DNA storage architecture

3.2.4 Majority Nucleotide Selection Algorithm

The similarity between one-to-many sequences is detected based on hamming distance with mismatch threshold. After that, clusters are formed with specified size of members in cluster. Representative sequence is reconstructed with fewer errors by selecting majority nucleotide in each position within members in cluster and random nucleotide is selected if majority is not observed [60].

3.2.5 Cooperative Sequence Clustering

First step [18] in this technique is to cluster the reads having Hamming distance of 2 to the same cluster. Then it's been sorted in the descending order of the cluster size. Second step is to discard the reads. For the groups of clusters with size 1, the discarding is performed by comparing Hamming distances to the combined sequences of the preceding clusters, and surviving groups of clusters with size 1 are sorted by the product of quality score values in descending order in the third step.

We investigated the suitability of Starcode, MeShClust, Slidesort, Majority nucleotide selection and Cooperative sequence clustering (as shown in Figure 8) for improving the performance in DNA data storage. The General Linear Model (GLM) model of MeShClust is modified and best fit of parameters are evaluated. The post processing algorithm is devised to enable the use of Slidesort for DNA data storage. Best clustering type is determined in Starcode and parametric tuning is performed in Majority nucleotide selection algorithm.

For Nanopore sequences, with respect to its high error rate and sequence length, the clustering algorithm which uses quality value 'isONclust' is used for clustering the simulated reads in coherence with its better performance.

3.2.6 isONclust

This algorithm [52] clusters the Nanopore sequences based on quality value using both exact matching algorithm through minimizer approach and approximate matching technique such as Smith-Waterman algorithm. Initially, the reads are sorted in descending order with higher quality values.

Sorting order is calculated based on the score $s(r)$ (as shown in Equation 3) of a read r as the expected number of error-free k -Mers in r . x_i is a binary indicator variable modelling the K -Mer starting at position i of r which has no sequencing error. The score of read can be computed by scanning linearly over a sliding window of k quality scores.

$$s(r) = \sum_{i=1}^{(|r|-k+1)} E(x_i) = \sum_{i=1}^{(|r|-k+1)} \prod_{j=0}^{k-1} (1 = q(r_{1+j})) \quad \text{----- Equation 3}$$

The representative sequence is determined as sequences having the largest number of error-free K -Mers among all other reads. Hash table $H(x)$ containing all minimizers for any k -Mer x is used for determining representative sequence. Recomputing of representative sequence on each update is avoided thereby saving time.

Subsequently, each member has to be moved to a different cluster. To do this, each read's minimizer is queried, and a similarity match is performed with the representative's minimizer. If the matching threshold is high. i.e., a high fraction of read's sequence shared with representative sequence (shared minimizer of at least 70%), then the read is grouped as a member under cluster having corresponding representative sequence. If a shared minimizer is not present or if the minimizer matching approach is prevented from detecting similarity, an alignment approach addresses those reads for clustering together. The fraction of read sequence shared with a representative is estimated and if the quality value is above the threshold, the read is assigned to the corresponding representative sequence [38]. Hence either using minimizer matching or alignment approach, all the reads are assigned to a cluster having a representative sequence. The quality value is prominent in each stage such as during sorting, deciding whether the mismatched minimizer is due to sequencing error, and also in computing pairwise alignment.

3.3 Material and Methods for Proposed Simulator

In-order to benchmark new methods for clustering and encoding/decoding techniques, empirical data is required which is sometimes difficult and expensive to obtain with unknown ground truth. However, the simulated data obtained via simulators can also be used to benchmark those methods. Hence, simulators play a prominent role in analysing the error distribution and designing a suitable technique for the encoding/decoding process and clustering.

3.3.1 Difference of Simulator Requirement in Biological and DNA Storage domain

In genomic sequencing, the long genome/DNA sequence is split into several overlapping and non-overlapping sequences. The genome splitting of varying lengths is determined by a factor called coverage (number of reads) and read length distribution. To economically study the read length distribution along with error distribution and error patterns, a biological simulator mimicking the sequencing process and generating approximated results is designed. But in the case of DNA data storage technology, coverage is specified as the ratio of the number of sequence readouts against the number of encoded original sequences which is different from that of the coverage termed in biological simulator tool. The simulators in the biological domain do not provide an option to customize the size of sequencing coverage.

Different simulators are used to simulate the process of Nanopore sequencer such as Readsim, Silico and Nanosim. As Readsim was specifically designed for PacBio sequencer, it does not mimic error distribution of Nanopore sequencer besides mimicking the approximate read length. Silico [61] also does not mimic error distribution of the sequencer exactly but it is provided with option to define the

insertion, deletion, and substitution rate. Nanosim [62] works based on Hidden Markov model where indel pattern, substitution pattern, and length distribution are being studied using empirical dataset. Predefined error model designed with earlier observed empirical dataset won't be a good fit as it varies because of advancement in basecaller from Albacore to Guppy. Basecaller does the conversion of signal into nucleotide sequence after Sequencing process.

Instead of designing the simulator by using the statistical concept, natural process of nanopore sequencer was simulated with Deep Simulator (Figure 9) and there is three component which is insulated from each other such as sequence generator, signal generator and basecaller. Deep simulator is a model which mimics the real nanopore sequencer providing the result in form of a signal and then basecalling it with standard basecallers for obtaining nucleotide sequences.

Owing to its approximation to the natural nanopore sequencing and various forms of results including both signals and sequences, DeepSimulator is preferred to be used in simulating DNA data storage. However, DeepSimulator simulates only the sequencing process but not the other steps of DNA data storage such as the process of the synthesizer and PCR where the number of sequences is amplified exponentially. Moreover, it does not provide any option to customise the data distribution based on sequencing coverage.

3.3.2 Components in DeepSimulator

3.3.2.1 Sequence generator

In DeepSimulator, the long input sequence is passed first to the preprocessing stage, in which several short sequences are sampled based on coverage and read length distribution. This preprocessing is modified to accommodate DNA data storage scenario where each input sequence is not sampled with certain short read lengths but with full length and specified coverage.

3.3.2.2 Signal generator (Pore Model)

The sequences obtained as output from sequence generator can be passed to signal generator module which contains pore model and signal repeating component. There are two pore models such as context dependent and context independent pore model. Context dependent pore model works based on bidirectional-Long Short-Term Memory (bi-LSTM) using trained biological dataset and Context independent pore model works based on official nanopore k-mer model. For DNA data storage, context-independent pore model is an appropriate fit as data could be customized and varied from traditional biological data. This pore model converts sequences into event sequences where each 6-mer is mapped with a current signal based on gaussian distribution with a specified mean and standard deviation.

The signal repeating component modifies the current signal/ event sequence into ground truth signal by matching it with the scale similar to real nanopore reads through repetition of signal at each position based on signal repeat time distribution which also adds noise to simulated signal. In-order to conquer difficulty of handling scale issue, Deep Canonical Time Warping (DCTW) method that handles temporal alignment and correlation analysis between current signal and DNA sequence based on real nanopore signal is calculated for several dataset and suitable distribution is estimated as mixture alpha based on approximate repeat time statistics [63]. Simulated ground truth signal is 8 to 9 times longer than event sequence.

The ground truth signal resembles the real nanopore signal by passing it to low pass filter and adding the gaussian noise. High frequency components embedded in the square waves are filtered out using low pass filter. In real nanopore sequencer, DNA strands are pulled through nanopore with the speed of 450 bp/s. Hence sampling frequency is fixed to 4KHz in deep simulator and cut off frequency is kept higher than 450 Hz. Higher cut-off frequency results in sharper signal with high base-calling accuracy whereas lower cut-off frequency results in smoother signal with less base-calling accuracy. Since raw signals are noisy due to sequencing environment, the gaussian noise are added at each position of simulated signal through parameter called 's'. In addition, the gaussian noise at event level can also be set with parameter 'e'. With this parameter 'e', the current signal is altered for same 6mer in different position along the sequence. When this noise-based parameter increases, the basecalling accuracy decreases.

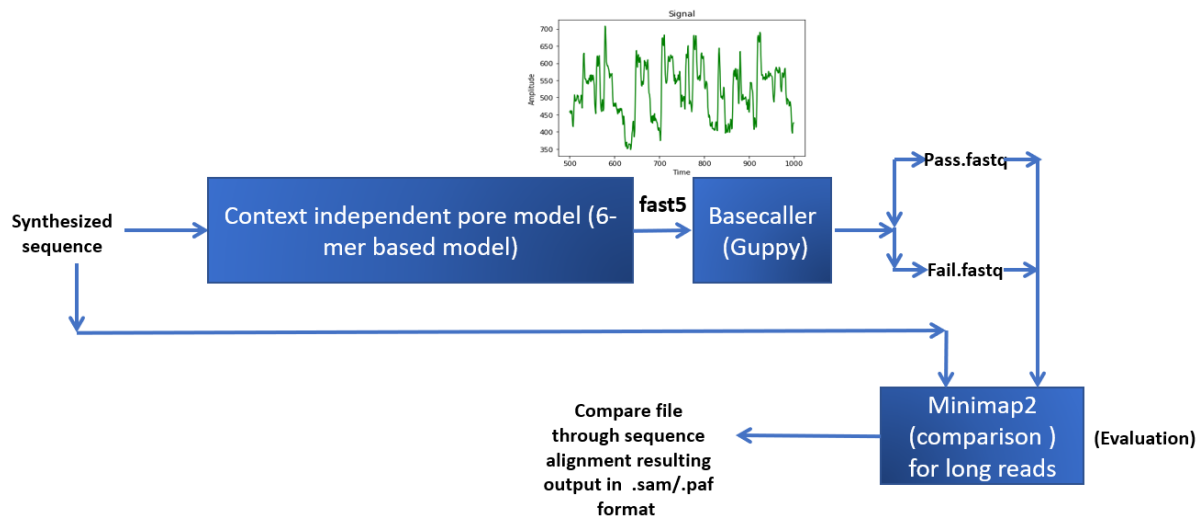


Figure 9: Pictorial Representation of Deep Simulator

3.3.2.3 Base callers

In Deep Simulator, the current signal is mimicked similar to Nanopore sequencer and the basecaller introduces mismatches and indels by itself. To overcome the errors leading to low signal-to-noise ratio due to structural similarity of nucleotides, non-uniform speed through the pore, homopolymer effect that occurs during the process of sequencing and basecalling, the nanopore chemistry and basecaller software has to be significantly improved. Basecaller uses Neural Network architecture where the network is trained on current signals obtained from real sequencing data.

Initially designed basecallers such as Nanocall, DeepNano, and Nanonet performs the pre-processing step of segmenting generated raw signal into discrete events. Nanocall uses the Hidden Markov model for predicting DNA sequence based on transition probability emitted from hidden states as k-mers [64]. The most probable path is computed through Viterbi decoding while basecalling. DeepNano uses Recurrent Neural Network (RNN) which predicts the event in both directions and combines them in the next layer [65]. And Nanonet uses Bidirectional RNN with Long Short-Term Memory units [66][67].

Chiron combines Convolutional Neural Network (CNN) and Bidirectional Long Short-Term Memory (Bi-LSTM) with Connectionist Temporal Classification (CTC) decoder [68]. Scrappie has two options with the event segmentation step and without that step. The transducer is included in the Scrappie module to improve homopolymer calling. Flappie has been replaced by Scrappie because of CTC to decode bases. Flappie uses the Flip flop algorithm to distinguish repeated bases. Albacore, Guppy, and Scrappie were developed with alternating reverse GRU architecture. Guppy, the recent basecaller also has RNN in its architecture and Sloika, neural network training toolkit used to make models for Guppy. It also implements a flip-flop algorithm in the basecalling model. Here in Deep Simulator, different basecallers can be integrated and used.

The basecaller results in three different outputs, including transition probability, position output, and basecalled sequence. Transition probability output comes from the neural network layer, position output file holds the position where signal repeats and basecalled output consists of DNA sequences. Different basecallers can be integrated and used in DeepSimulator. We have integrated Flappie basecaller for simulator validation since the real nanopore signals and basecalled sequences for comparison are from [31] where the research group used flappie basecaller. Moreover, a 10.5% error rate similar to a real nanopore signal is maintained.

In the proposed simulator, we designed the synthesizer and PCR module followed by subsampling of sequenced data using the non-parametric subsampling method (verified by Welch t-test) by studying the distribution of actual nanopore DNA storage data. After that, we integrated the proposed modules with DeepSimulator. Overall, we devised a tool depicting the DNA data storage channel of writing, storing, and reading data with nanopore sequencing. Moreover, we validated the generated raw signals

and their copies with original nanopore signal output by finding out sample differences through correlation. As the developed simulator has a feature of specifying coverage, it aids to quantitatively assess the association between *physical and logical* redundancy for systems with different experiment settings and code design, which helps in exploring optimal code and system design that balance the trade-off between *logical and physical* redundancy.

Chapter 4 Comparative analysis of Illumina based Clustering tools

Dataset 1 from [60] is utilized for conducting the experiment. It consists of 10,750 input encoded DNA sequences and 5812000 sequences read from sequencer. To analyze how the coverage affects the recovery rate in each algorithm, the total reads are split into varied numbers of sequenced read such as 11624, 23248, 34870, 46492, 58113, 69733, 81353, 92973 representing coverage varying from 1.081 to 8.649. Along with Majority nucleotide selection algorithm as implemented in [60], we are also experimenting with other algorithms for determining the better algorithm based on certain parameters.

Discussed algorithms such as Slidesort, MeShClust, Majority nucleotide selection algorithm and Starcode are experimented with dataset by modifying or changing certain parameters and the recovery rate is recorded for varied distances and coverage values. As Levenshtein/Edit distance increases from 1 to 8, the number of clusters formed decreases and the members inside the cluster increases. This results in a decrease of cluster representatives and an increase in the accuracy of cluster representatives (against encoded sequences). On comparing the cluster representatives with original 10750 sequences, the recovery rate can be computed. Besides, the coverage required by each algorithm to achieve a specified recovery rate is investigated to compare the tools in terms of sequencing performance measure. Moreover, the decoding complexity after applying each algorithm is evaluated based on the number of representative sequences after clustering. And the simulated ground truth distribution from [32] is compared with distribution generated after clustering to validate the grouping of clusters.

Similarly, datasets (dataset 2 and 3) from [18] is utilized for conducting the experiment so that the best clustering mechanism that can be applied for all datasets can be determined. Experiment uses 18,000 input encoded DNA sequences and 14 million sequences from constrained pool (dataset 2) and 20 million sequences from non-constrained pool (dataset 3) are read from sequencer. Here, the total reads are split into varied numbers of sequenced read such as 22797, 45594, 68391, 91188, 113985 representing coverages varying from 1.266 to 6.332. In these datasets, along with cooperative sequence clustering as implemented in [18], we are also applying other clustering techniques such as MeShClust, Majority nucleotide selection algorithm and Starcode for determining the better clustering tool.

4.1 Post Processing in Slidesort

Slidesort outputs only similar pairs out of the input data. Hence, we proposed a post processing module on top of it consisting of two steps including forming clusters which will group multiple sequences together (along with the removal of the redundant sequences) based on the edit distance and the estimate of the representative sequence in each cluster through mean shift algorithm. All edit distance pairs lower than the edit distance specified should be considered for clustering purpose. Slidesort outputs sequence 'a', 'b' paired with specified edit distance d as (a,b,d) . It can be illustrated by an example in which

Slidesort produces paired output as (1,2,0), (1,4,1), (1,3,2), (2,3,3), (2,5,3). Then the mentioned two steps will be applied for varied edit distance settings.

The Slidesort algorithm with its post processing module and mean shift algorithm is experimented with varied coverage size. After that, the representative sequences are compared with the original encoded sequences to obtain the recovery rate (Figure 10).

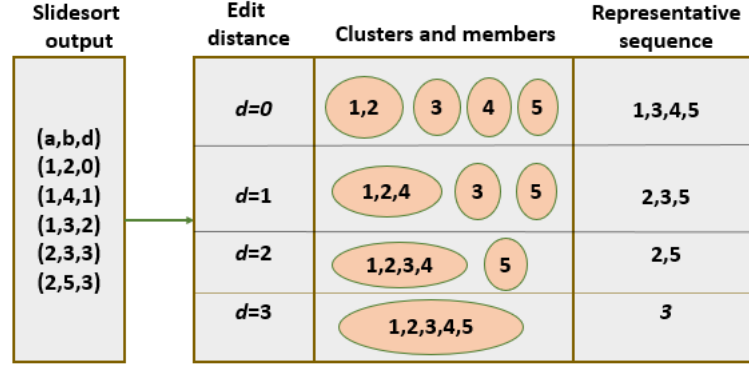


Figure 10: Proposed Post Processing in Slidesort

4.2 Modified GLM in MeShClust

As described in section 3.2.3, there are two modes, i.e., alignment and GLM based approach, to classify the sequences that are similar and dissimilar before clustering and estimating representative sequence using mean shift algorithm. As GLM based approach for classification increases speed compared to alignment, we are motivated to use GLM. But accuracy rate is less compared to alignment. Overall, 4 feature set used for training GLM with combination of sequence length difference, Czekanowski similarity, Manhattan distance, Pearson coefficient and Kulczynski distance and they are varied in each iteration to yield high accuracy. These features are extracted after converting sequences into k-mer histograms and GLM is trained and tested with these feature sets.

We have used datasets (1,2,3) and as observed in Figure 11, the default non-aligned GLM based approach provides the random recovery rates for varied identity scores. Hence, we analyzed the internal model and modified the model to yield comparable accuracy similar to alignment-based approach. In the default model, the minimum feature is an internal parameter, and it is set to 2. If it doesn't attain 97.5% accuracy, then the rest of the two features are added in the subsequent training process to improve the accuracy. Reducing the minimum feature parameter to 1 prevents overfitting and improves the reliability and performance of the trained classifier as 3 parameters will be available in consequent training step to improve accuracy. The implementation with alignment and with modified minimum features parameter in GLM can reach similar recovery rates as shown in Figure 11 for the three datasets.

Moreover, MeShClust is improved in its processing time with modified (minimum feature set) non-aligned GLM based approach from 292 minutes to 46 seconds yielding same accuracy as aligned method.

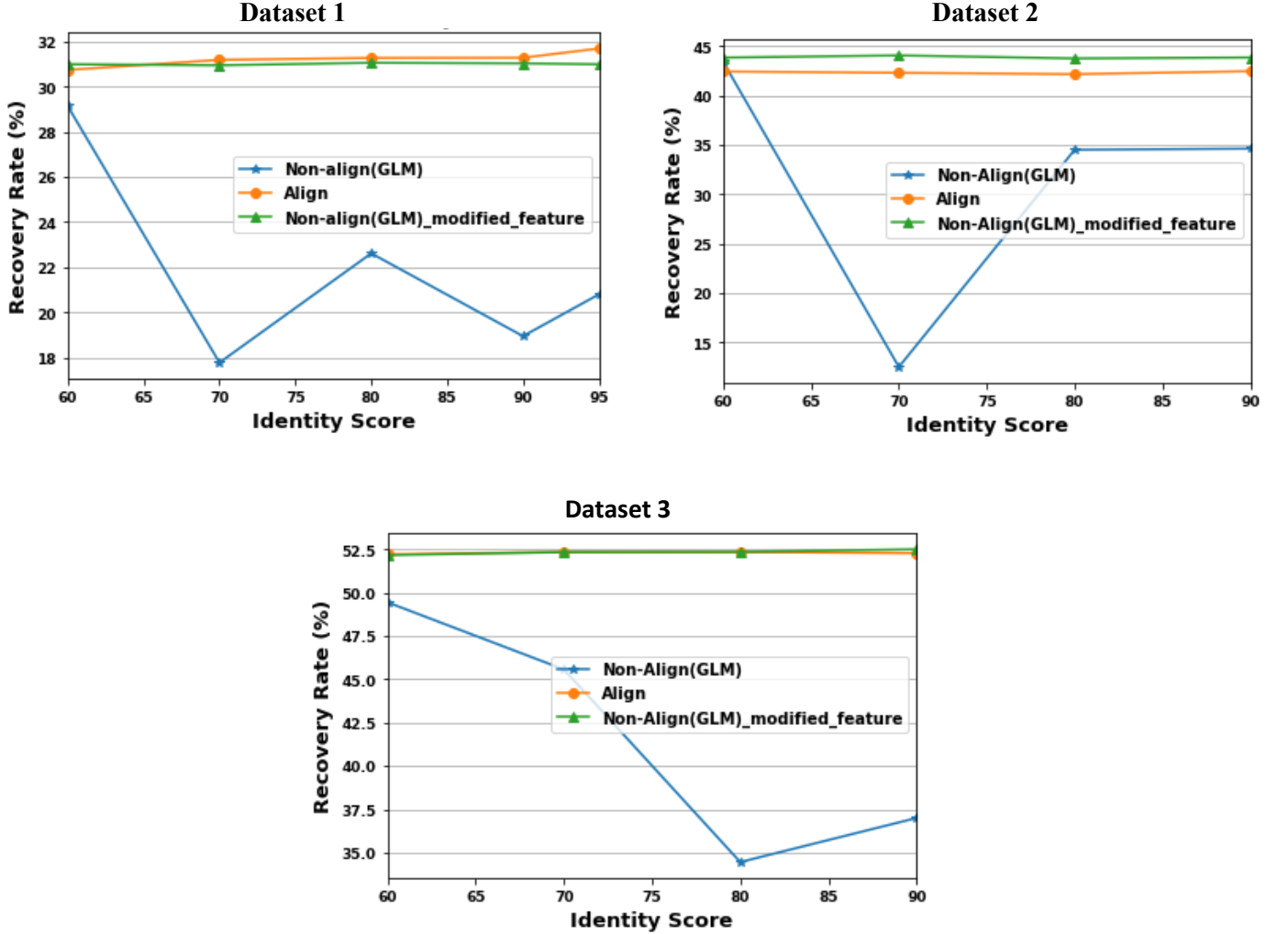


Figure 11: Comparison of recovery rates in MeShClust with alignment, non-alignment (GLM) and modified GLM

4.3 Tuning and Comparative analysis of all algorithms

Starcode is applied on datasets with varied coverage, and it matches all sequences within a specified Levenshtein distance and then clusters them using Message passing or Sphere clustering. In Starcode, for user specified edit distance $d_e \geq 1$, the algorithm will check if a data appears at least 5 times. If it is so, then it will cluster it. This operation is performed through parameter called clustering ratio. If a cluster has only one member, then that member will turn as a representative sequence with cluster size as one. With Sphere clustering method, for a fixed coverage, the clustering converges to a very low

cluster number for d_c equal to 8 and the recovery rate decreases for higher d_c value. Though it reduces decoder complexity owing to lower cluster number, it is not an optimum model. Hence message passing is chosen which provides results in the same recovery rate for varying values of d_c and fixed coverage with reduction in decoder complexity.

Majority nucleotide selection algorithm is tuned to different hamming distance d_h from 1 to 8 and clusters are formed with cluster size greater than 2. In addition, sequences which is not clustered are also considered as individual cluster for estimating the recovery rate. The tuned and customized algorithm is experimented with varied coverage sizes and d_h values followed by the estimation of recovery rate and we observed a high variation in recovery rate with respect to different coverage.

Table 1: Parameter Tuning

Parameter	Tuned Value	Selected
Identity	0.6 to 0.8	0.8
K-mer	1 to 8	4
Delta	1 to 10	4

In MeShClust, the most important external parameters such as identity score, k-mer and delta value are selected subsequently by implementing a modified non-alignment GLM technique on a set of sequences (Table 1). The results of our experiment prove that the MeShClust is not that sensitive to the identity score parameters in the modified GLM based approach as shown in Figure 11. Small K-mer value cannot extract enough characteristics on the input sequences and high K-mer value increases computational time. Hence, a K-mer value is selected, which falls in a relatively stable region and does not require much computational time. Besides, delta value specifies the number of clusters to be merged nearby after applying mean shift algorithm. The tuned parameters are used in determining recovery rate for MeShClust

To this end, we investigate the effectiveness of clustering by calculating the recovery rate, analyzing decoding and sequencing complexity. As shown in Table 2 and Figure 12, Starcode, Cooperative and Majority Selection algorithm lies at top with higher recovery rate provided the coverage is 8.6x, 6.3x, 6.3x for dataset 1, 2, 3, respectively. Then comes Slidesort and MeShClust having very less recovery rate. The reason behind using high coverage for dataset 1 might be the design with Single PBS [60]. Another observation is for constrained dataset (dataset 2), Starcode provides higher recovery rate of 93.40% than Cooperative providing 80.49% whereas each clustering methodology provides comparable accuracy for non-constrained dataset 3.

Table 2: Recovery rate by applying clustering algorithm on 3 datasets

Clustering algorithm	Recovery rate of dataset 1(Coverage: 8.6x)	Recovery rate of dataset 2(Coverage: 6.3x)	Recovery rate of dataset 3(Coverage: 6.3x)
Starcode	89.89	93.40	98.10
Cooperative	(Not calculated because of lack of q value)	80.49	98.11
Majority nucleotide selection	87.44	74.33	97.83
Slidesort	60	(Not calculated as it consumes lot of time)	(Not calculated as it consumes lot of time)
MeShClust	50	54.10	60.29

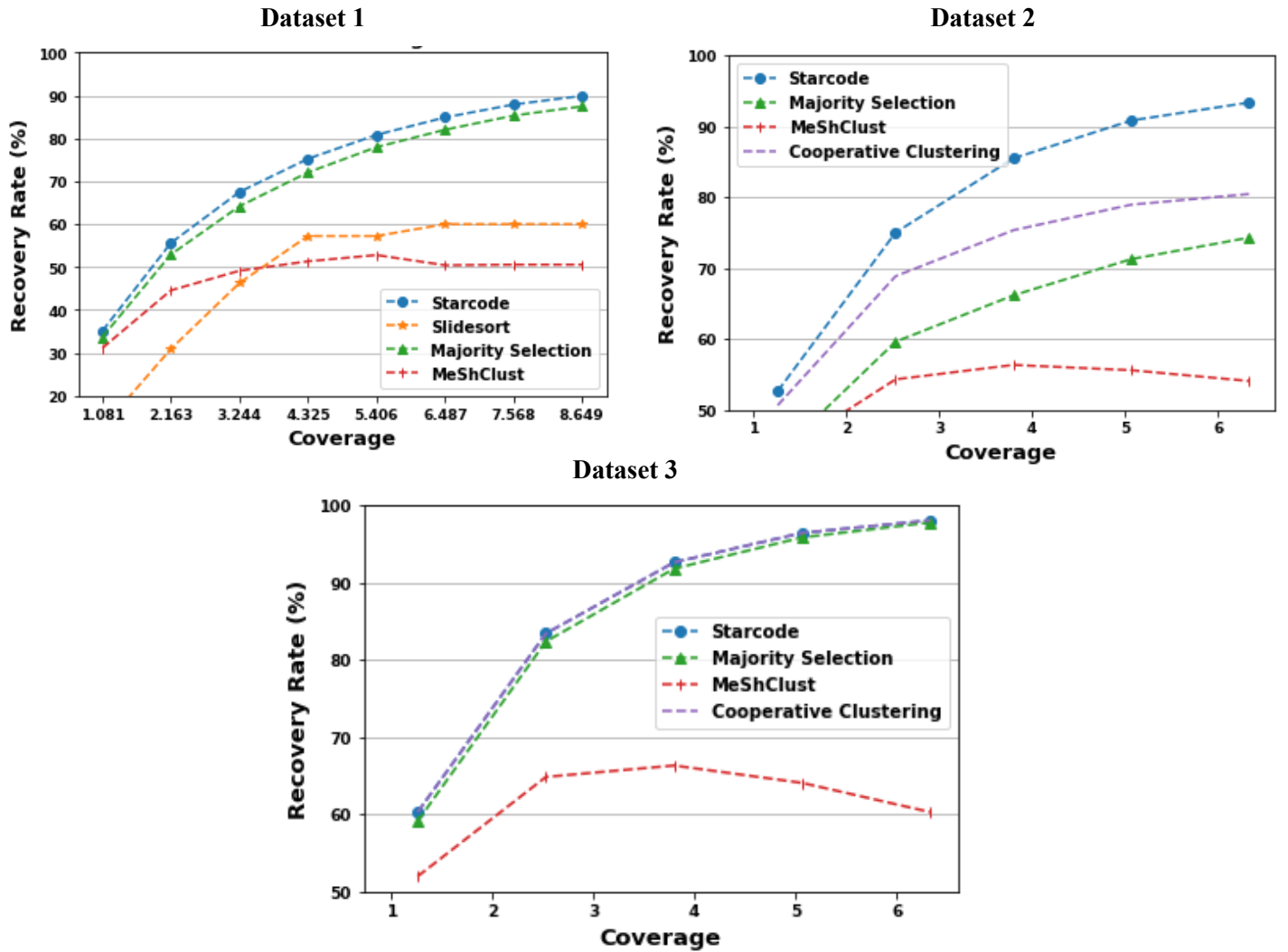


Figure 12: Comparison of recovery rates calculated from different tools

Moreover, it shows that to obtain recovery rate of 80% in dataset 1, Starcode requires coverage of 5.4x while Majority nucleotide selection requires 0.6x more and other two tools (Slidesort and MeShClust) do not achieve 80% of recovery rate even with 8.6x coverage. In dataset 2 to obtain same recovery rate of 80%, Starcode requires coverage of 3x while cooperative requires 5.5x and tools such as majority nucleotide selection and MeShClust do not achieve 80% of recovery rate even with 6.3x coverage. In dataset 3 to obtain higher recovery rate, Starcode, Cooperative and majority nucleotide selection requires coverage of 2.2x while MeShClust do not achieve 80% of recovery rate even with 6.3x coverage. This observation implies that Starcode and cooperative is the best to provide good recovery rate even with low coverage. In non-constrained dataset 3 and dataset 1, Majority nucleotide selection also provides good recovery rate.

In terms of speed, Starcode-message passing takes 15 minutes for 8x coverage, but Majority selection algorithm takes around 2 hours. And for 6x coverage, cooperative sequence clustering takes around 15 minutes. Note that for performing Starcode written in C, we used Intel Haswell 2 vCPUs with memory of 4GB whereas for Majority nucleotide selection algorithm (MATLAB) and Cooperative Sequence clustering (C++), Intel core i5 processor with memory of 8GB is used. With further code optimization and computation environment improvement, the Majority nucleotide selection method could be sped up to compete with Starcode.

In terms of decoding complexity (Figure 13), Slidesort produces best result as number of clusters converges. However, it is not an optimum fit because of its relatively low recovery rate. Majority nucleotide selection also presents good decoding complexity compared to Starcode in all datasets. Thus, we conclude that Majority nucleotide selection is a better algorithm in terms of yielding comparable recovery rate and reducing decoding complexity. Note that MeShClust cannot be compared as the tool is not designed to tune the distance parameter. Cooperative sequence clustering uses fixed hamming distance of 2 and provides less complexity with cluster numbers formed 22335 and 17987 for constrained dataset 2 and non-constrained dataset 3 respectively which is comparable with majority nucleotide selection clustered by tuning hamming distance to 2.

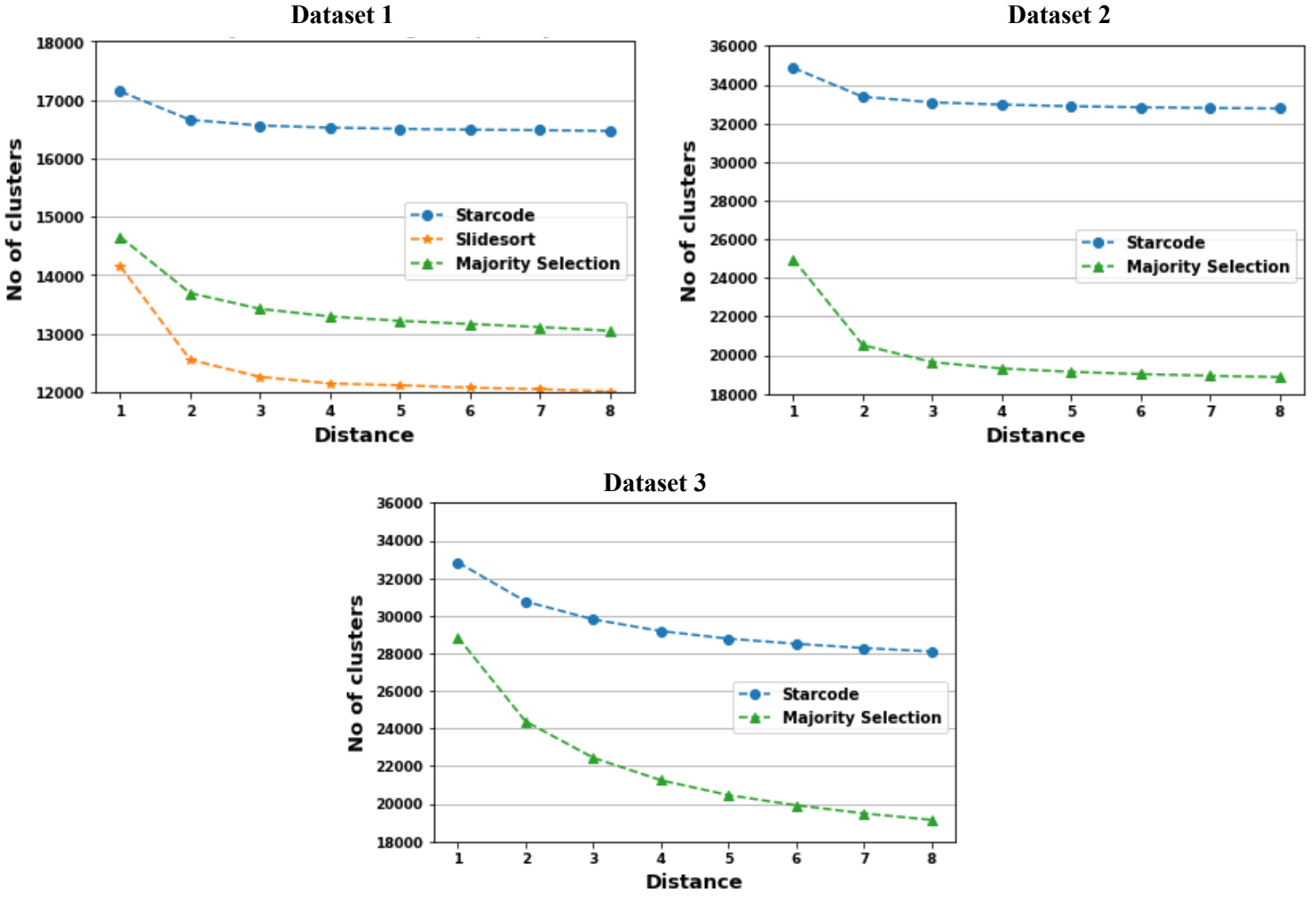
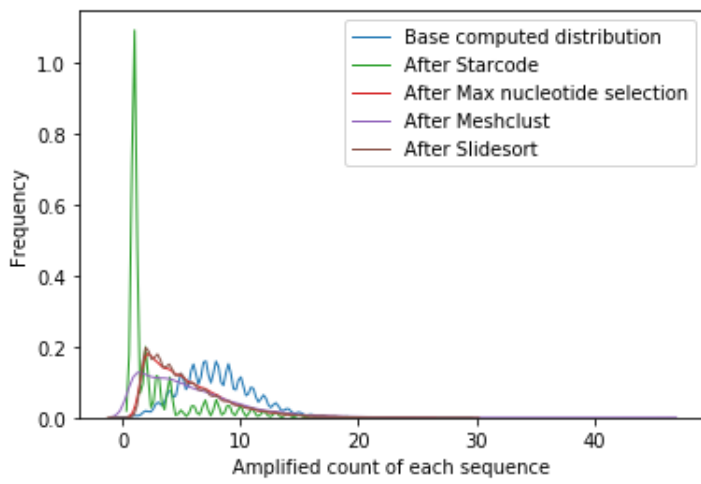


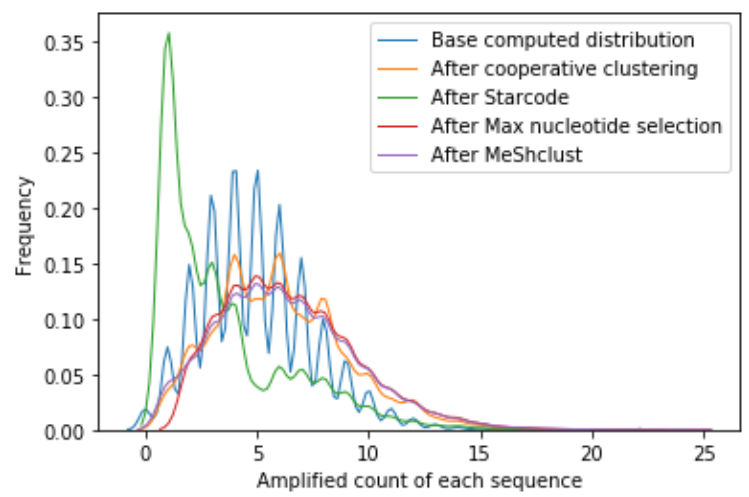
Figure 13: Comparison of decoding complexity, i.e., the number of clusters after applying clustering mechanism

Other evaluation measure such as clustering distribution for the purpose of determining how well can the tools assemble is estimated. Ground truth distribution (specified as base computed in graph) is simulated by specifying higher coverage such as 8.4x for dataset 1 and 6.3x for dataset 2 using statistical computational method from paper [32]. Starcode consist of many clusters with 1 cluster element for all datasets. And all other tools try to reproduce the base computed for dataset 2 and 3 and there is a drift in case of dataset 1 (as shown in Figure 14).

Dataset 1



Dataset 2



Dataset 3

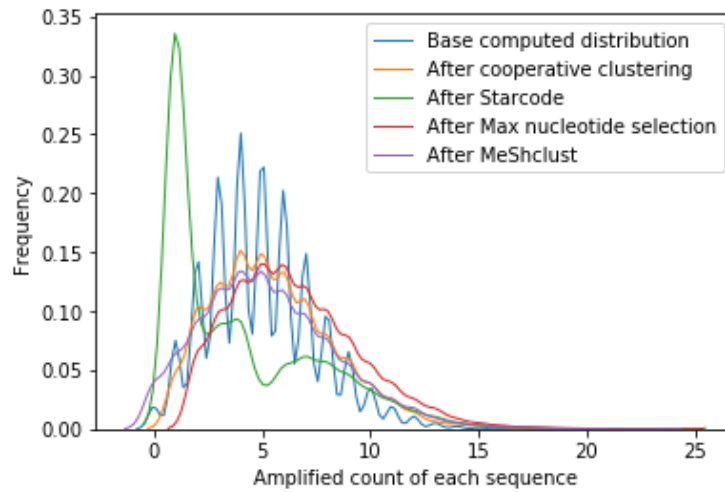


Figure 14: Comparison of distribution between ground truth (base computed distribution) and after clustering

4.4 Summary

This comparative study aids in selecting suitable clustering techniques for downstream processing (or reconstructing) of DNA storage data before decoding. Minimal coverage and the maximal recovery rate are the keys in determining the best clustering tool thereby reducing sequencing complexity. In addition, less decoding complexity and predicting ground truth distribution are the other two keys. We observed that Starcode and Cooperative clustering performs best in terms of reducing sequencing complexity. Starcode, Cooperative, and Majority nucleotide selection perform best in non-constrained datasets whereby it uses minimal same coverage to achieve a high recovery rate. Slidesort is ahead of other tools in terms of reducing decoding complexity though it gives a poor recovery rate and Majority nucleotide selection lies in the next position of reducing decoding complexity. In terms of distribution, Starcode doesn't achieve the ground truth whereas the rest were able to do. Hence based on the above analysis maximum nucleotide selection and cooperative clustering provides a good result for all dataset types. Starcode achieves the best result but at the cost of increasing decoding complexity though d value is specified remarkably high as 8.

Chapter 5 Design of Computational Simulator for DNA storage (based on nanopore sequencing)

We propose a DNA data storage simulator for generating copies of each sequence obtained after undergoing synthesizing, amplifying, and sequencing process based on data distribution (Table 3). In addition, indels and substitution are embedded with specified rates, which simulates errors appearing in the synthesizer process. Using the non-parametric sub-sampling method, we estimated copy count distribution of the encoded sequences after sequencing and sampling, and errors are randomly embedded into these copies according to the synthesis error rate. Six steps followed in the design of the simulator are

- Detection of distribution
- Statistical analysis
- Implementation of copy distribution for various mean coverage values
- Computing Confidence Interval (CI) and Coefficient of Variation (CV) for all distributions
- Integration of Preprocessing module in DeepSimulator
- Validation of Proposed Simulator
- Nanopore based clustering tool: isONclust

5.1 Detection of distribution

Chandak.et.al [31] designed coding methodology with the outer code as Reed Solomon and inner code as Convolutional code. The encoded data consisting of varied number of sequences in each experiment and are mapped to oligonucleotides. Then all sequences of varied experiment are merged and synthesized along with PCR and sequenced for specific experiment. For analysis purpose, the sequenced read is mapped with reference id. This can be used for finding out distribution of the sequenced data for each experiment. The real time signal data obtained from Nanopore sequencer after custom array synthesis, and PCR is used for finding the distribution of data. With these data obtained from 10 experiments, analysis is carried out to determine the probabilistic model associated with them. Mostly, the distributions observed in 10 experiments are normal, beta and log gamma as shown in Table 3. These distributions are modelled and integrated in the deep simulator along with synthesis process where indels and substitution errors are embedded in sequences. Best distribution can be identified from several criterion such as Sum square error (SSE), Akaike information criterion (AIC), Bayesian information criterion (BIC). Sum of the squared differences between observation and its group's mean is called SSE. Error prediction and relative quality of each distribution can be evaluated by a factor called AIC and best model should have lower AIC. Another criterion for selecting the best model is through BIC where best model should have low BIC. Specifically, when statistically fitting the model to

data, adding more parameters may increase the likelihood, but it may also result in overfitting. AIC and BIC measures resolve this by introducing penalty terms for the number of parameters in the model; Penalty is higher in BIC than AIC. When the number of observations is large, the AIC and BIC will produce different fitting of model results. But as BIC applies larger penalty for complex models, it will lead to correct fit than AIC. Hence in our detection algorithm, based on first priority of BIC, second priority of SSE and then finally AIC, the best model is determined. For modelling those distribution after evaluating the best probabilistic model, two bootstrap sampling methodology namely parametric and non-parametric sampling is compared, and better method is determined.

Table 3: Distribution of all experiments

Exp	Code memory (m)	Code memory	Outer code Redundancy	Encoded data(A)	Sub sample after sequencing (B)	A compared with B (C)	A-C (oligo loss)	Mean	Std	Detected Distribution	Sum square error	AIC	BIC
0	8	1/2	0.3	1466	20000	1383	83	13.642	9.244	Skew norm	0.224	439.9	-12862.12
										Log gamma	0.242	631.8	-12745.00
										Norm	0.243	639.8	-12746.54
1	11	1/2	0.3	1466	10000	1442	24	6.821	3.871	Norm	0.014	174.7	-16898.35
										Log gamma	0.015	178.6	-16769.17
										Log norm	0.018	144.9	-16559.53
2	14	1/2	0.3	1466	10000	1419	47	6.846	4.383	Log norm	0.036	385.6	-15523.85
										Norm	0.037	733.5	-15487.89
										Gamma	0.044	400.4	-15254.41
3	8	3/4	0.3	815	20000	810	5	24.540	14.846	Log gamma	0.382	947.6	-6227.02
										Norm	0.389	956.1	-6217.94
										Beta	0.445	714.0	-6096.59
4	11	3/4	0.3	815	10000	807	8	12.270	8.434	Skew norm	0.095	471.16	-7359.82
										Beta	0.128	454.79	-7110.94
										Gamma	0.138	414.91	-7054.66
6	8	5/6	0.3	733	20000	733	0	27.285	20.894	Log norm	0.734	1024.5	-5042.83
										Gamma	0.900	1121.6	-4893.59
										Beta	0.912	1135.8	-4876.85
7	11	5/6	0.3	733	10000	731	2	13.642	5.736	Skew norm	0.022	245.25	-7585.48
										Log norm	0.023	242.79	-7580.49
										Beta	0.024	245.94	-7552.17
8	14	5/6	0.3	733	10000	724	9	13.642	7.851	Norm	0.124	401.35	-6349.07
										Log gamma	0.125	404.11	-6338.13
										Log norm	0.162	317.12	-6147.62
9	11	3/4	0.2	752	10000	744	8	13.297	8.018	Beta	0.145	248.45	-6407.16
										Norm	0.162	288.30	-6333.48
										Log gamma	0.163	292.58	-6324.91
10	11	3/4	0.4	877	10000	864	13	11.402	7.572	Beta	0.094	323.84	-7988.67
										Skew norm	0.118	306.10	-7792.57
										Gamma	0.123	292.64	-7757.77

5.2 Statistical Analysis

Bootstrap sampling is a statistical computational method to sample data from large set of samples by making use of its statistical parameter or through sampling with replacement. It describes how estimates from varied experiments are distributed and precision as well as bias can be determined. This is used for simulating samples from large sample set and then standard errors as well as confidence interval can be calculated along with hypothesis testing. The population distribution can be estimated from the observed samples. There are different forms of bootstrap sampling such as Non-parametric, Semiparametric and Parametric. Non-parametric bootstrapping is resampling the data with replacement from large set of samples and inferring the statistical estimate from the resampled data. It has good statistical properties. In Semiparametric bootstrapping, sampling with replacement is performed similar to non-parametric bootstrapping and then noise is added. In parametric bootstrapping, the estimated distribution along with estimated parameter creates the samples.

To evaluate the accuracy of statistical estimate using parametric and non-parametric bootstrap sampling, the confidence interval can be calculated by conducting experiments with several iterations. Confidence interval tells us the deviation of sample estimate from the true value. CI (Confidence Interval) helps in enhancing the model by making it closer to the actual sequencing process. The samples are randomly sampled B times with size n from P dataset or from estimated distribution along with distribution parameters. Statistics such as mean, and variance is computed for each sample set. Now, overall mean and variance of these B statistic is computed which converges to true mean and variance of P dataset as shown in Figure 15. For constructing the confidence interval, the standard error method is preferred compared to percentile method as the distribution resembles close to normal. To determine the best sampling method, we evaluated the sampling model through hypothesis testing, histogram, distribution analysis, CI, mean value range (as shown in Figure 16)

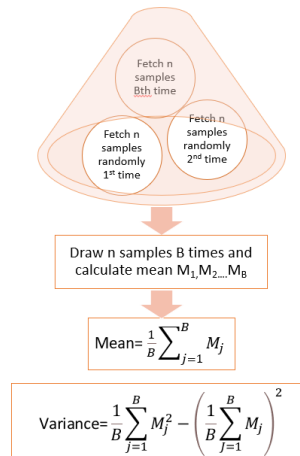


Figure 15: Bootstrap Sampling

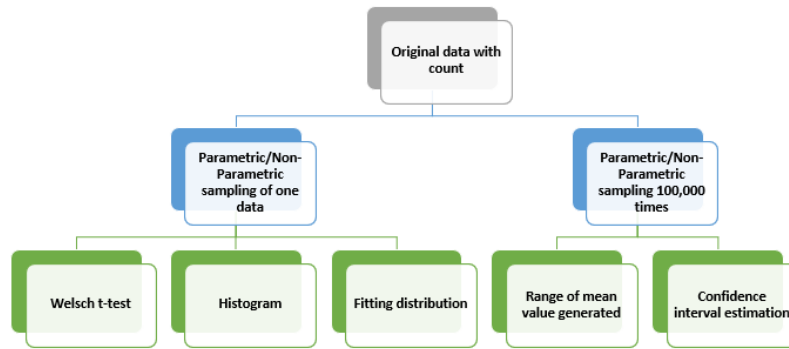


Figure 16: Statistical analysis for evaluating sampling methodology

Using the data available from the experiment, we carried out the non-parametric and parametric sampling methods and performed Welsch t-test along with histogram analysis of copy distribution. The statistical test is performed to test the hypothesis of whether the two populations (original and sampled output) have equal means. Welsch t-test is an adaptation of student t-test for samples with unequal variance and sample size.

$$t = \frac{(\bar{x}_1 - \bar{x}_2)}{\sqrt{\left(\frac{s_1^2}{N_1} + \frac{s_2^2}{N_2}\right)}} \text{ -----Equation 4}$$

$$v \approx \frac{\left(\frac{s_1^2}{N_1} + \frac{s_2^2}{N_2}\right)^2}{\frac{s_1^4}{N_1^2 v_1} + \frac{s_2^4}{N_2^2 v_2}} \text{ -----Equation 5}$$

Where $\bar{x}_1$ and $\bar{x}_2$ represent the means of original and sampled data, N_1 and N_2 represent the numbers of original and sampled data, s_1 and s_2 represent the Standard deviations of original and sampled data, v_1 and v_2 represent the degrees of freedom from original and sampled data $v_1 = N_1 - 1$ and $v_2 = N_2 - 1$

Table 4: Student t-test statistical result

	Original samples	Parametric samples	Non-parametric samples
Mean	6.821	7.085	6.867
Standard deviation	3.871	3.520	3.776
Sample size	1466	677	677
t-statistic with original samples		-1.565	-0.259
p-value		0.12	0.79

From Table 4, it could be found that the non-parametric method has higher p-value (0.79) than the parametric method (0.12) and both are not statistically significant ($P > 0.05$), indicating strong evidence

for the null hypothesis. However non-parametric samples are very close to original samples than parametric.

We have drawn the histogram of original and simulated samples (Figure 17) for parametric and non-parametric sampling models. The histograms look similar, but the fitted distributions vary. We observed that normal distribution, which is the distribution of original sample data of experiment 1 stands first in non-parametric based on model selection criterion. And in parametric samples, log norm is detected at the first position. This again proves the advantage of using non-parametric over parametric to approximate the copy count distribution.

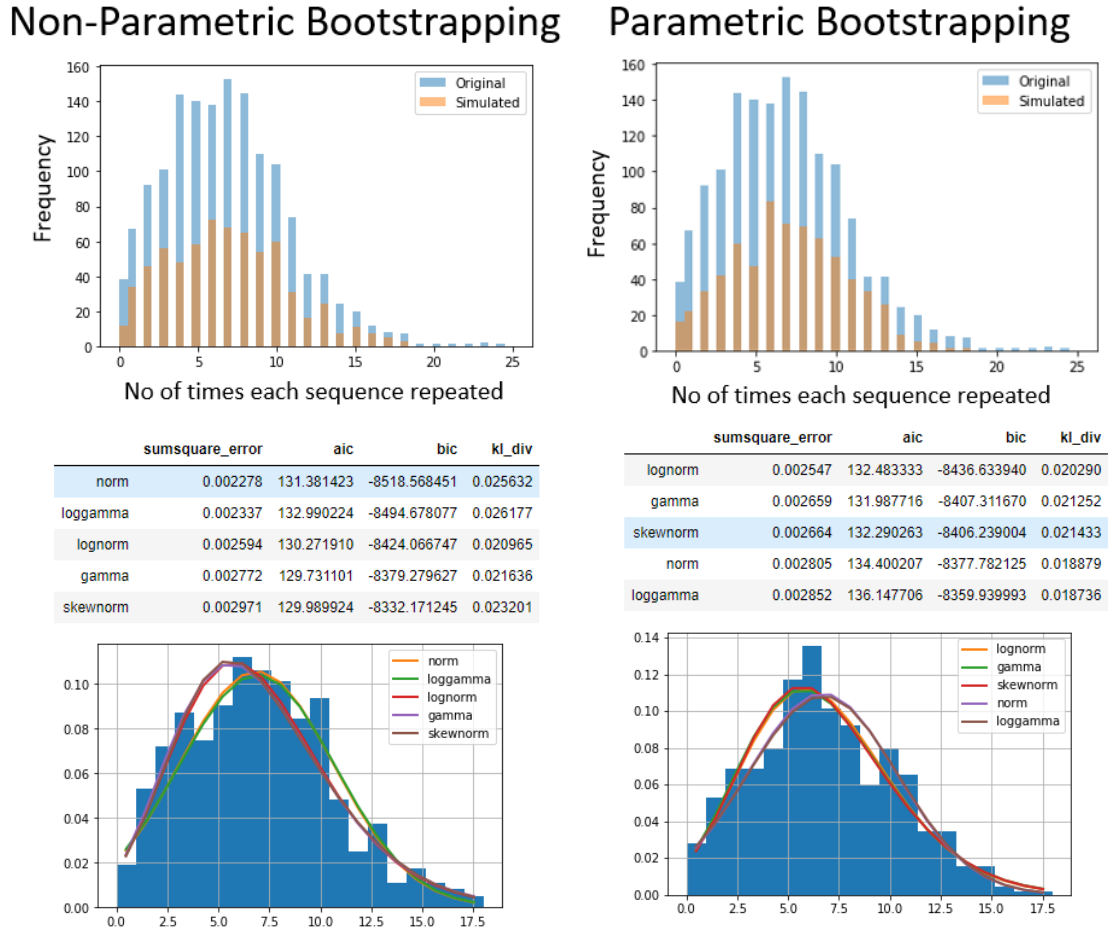


Figure 17: Histograms of sequences and distribution

We carried out further analysis to determine the better sampling method through CI and mean range estimation. Original distribution (experiment 1 of Table 3) has mean and coefficient of variation (CV) of 6.821 and 1.7621. Now, we performed sampling through parametric and non-parametric methods 100,000 times. Out of that, we selected the best sample set consisting of count values by finding out the range of mean with its tuned CI and subsequently filtering by fixing the simulated CV to CV of original data rounded to 4 digits, i.e., [1.7621, 1.7622]. As a result, from the filtered dataset, we randomly picked one dataset.

Setting higher CI leads to filtering a sample set out of the wider sample sets, whereas lower CI leads to filtering a sample set out of the narrower sample sets. From this idea of setting up confidence interval, it can be inferred that the best sampling method should work even on lower confidence interval.

Through trial-error tuning experiments, we identified that non-parametric bootstrap sampling works with 90% CI and a mean range of [6.6733, 6.9696] and parametric bootstrap sampling works with 95% CI and a mean range of [6.8158, 7.3589] (Table 5). If the confidence interval is narrowed further in parametric bootstrap, the next filtering step using coefficient of variation is aborted which means parametric bootstrap sampling requires a greater number of bags sampled to determine the best sample. This decreases the probability of getting samples with estimated statistic similar to true statistic on a single sampling process.

Table 5: Confidence interval tuning using standard error method

	Non-parametric	Parametric
Confidence Interval	90%	95%
Mean range formula calculated from confidence interval	[Mean - std, Mean + std]	[Mean-(1.96*std), Mean+(1.96*std)]
Mean range Original mean: 6.821	[6.673278, 6.969596]	[6.815854, 7.358944]
Coefficient of variation range, Original C.V: 1.762123	[1.762100, 1.762199]	[1.762100, 1.762199]

As shown in Figure 18, original mean values of 6.8 is centralised in case of non-parametric fitting and mean value shifted to right in parametric fitting. This means that the same mean value as the original is generated more times in non-parametric sampling, proving non-parametric fitting performs better in terms of approximating to the real statistic.

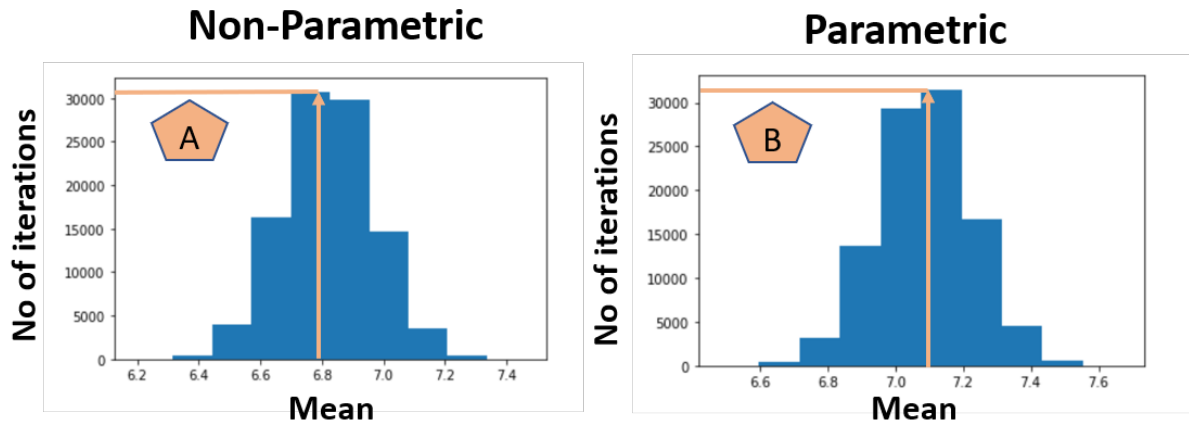


Figure 18: Interval of mean. Comparison of original mean with simulated mean in different sampling methods. Observations: A - For Maximum iteration, the observed mean is equal to original mean, B - For Maximum iteration, the observed mean is greater than original mean

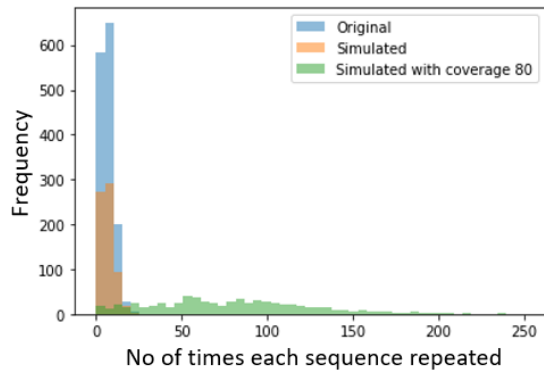
5.3 Implementation of Copy Distribution for various Coverage values

Increase of coverage value has to be addressed as it plays vital role in DNA channel model when it comes to sequencing. For instance, as per the experiment 1 shown in Table 3, the coverage is 6.8x for randomly extracted dataset (obtained by evaluating the redundant copies of experimental data). But the proposed algorithm must be able to sequence more/less copies too with coverage greater than or lesser than 6.8x. Hence sampling with replacement is done based on the probability of occurrence of each sequence if coverage is varying from 6.8x. Simulated copies and the original distribution are shown in histogram (Figure 19).

After approximating the copy count distribution from the real dataset with certain coverage, we equipped the simulator with an option to generate copy distribution with various coverage values. In this case of higher coverage requirement, the experiment is conducted 1000 times, the coefficient of variation is tuned and non-parametric provides less variation on tuning such as [1.760, 1.769] whereas parametric leads to higher variation such as [1.76, 2].

Lessening the coverage value than original coverage value/mean should also be addressed. In this case also, the experiment is conducted 1000 times and filtering is done through tuned range of coefficient of variation and it results in less variation for non-parametric of [1.20, 1.77] and high variation for parametric of [1.00, 1.77] as shown in Figure 19.

Non-Parametric Bootstrapping



Parametric Bootstrapping

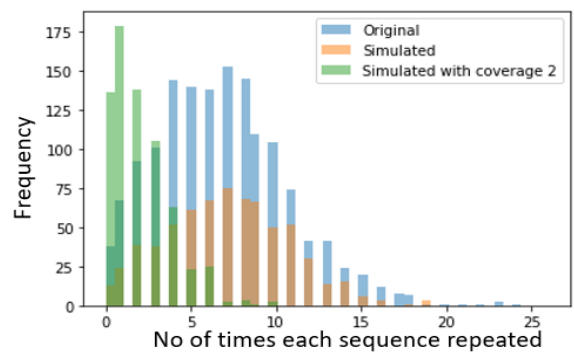
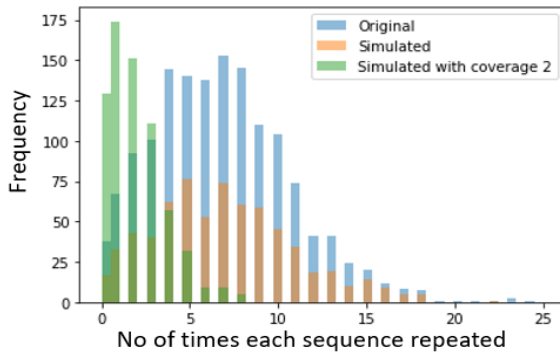
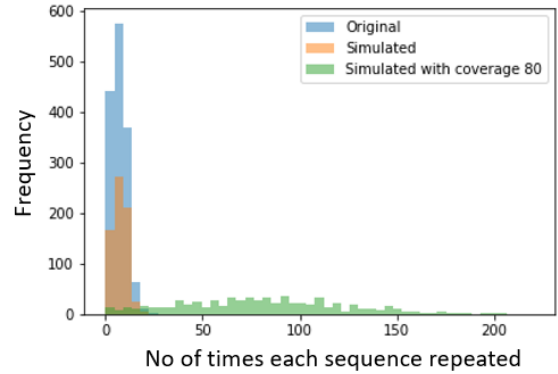


Figure 19: Histograms for different coverages

On inferring results from various analysis, non-parametric bootstrap sampling is the best because of various reasons such as p-value, drift of mean range, higher changes in coefficient of variation compared to the original, and lower confidence interval.

5.4 Preprocessing module and Computing Confidence Interval and Coefficient of Variations for all distributions

Based upon the previous findings, we proposed a pre-processing module as shown in Figure 20. From the real-time hdf5 file, 10,000 samples were extracted and the repetition of each encoded input sequence in sample set A was recorded. If an encoded sequence is not mapped with any of the samples, it is termed as lost with 0 count in sample set A. The non-parametric bootstrap sampling is iterated 10,000 times from sample set A for extracting specific sizes of samples. As a result, we got 10,000 estimated sample sets A1, A2...A10000 with a specified sample size (677 which is approximately half of 1466 samples of experiment 1) consisting of different count values. First, the mean of these sets is computed, yielding 10,000 means. Next, we averaged the resulting 10,000 means and calculated the standard deviation value. Then, with those outputs and setting 90 as percentile using standard error method, we computed the mean range through CI calculation for all the distribution (Figure 21).

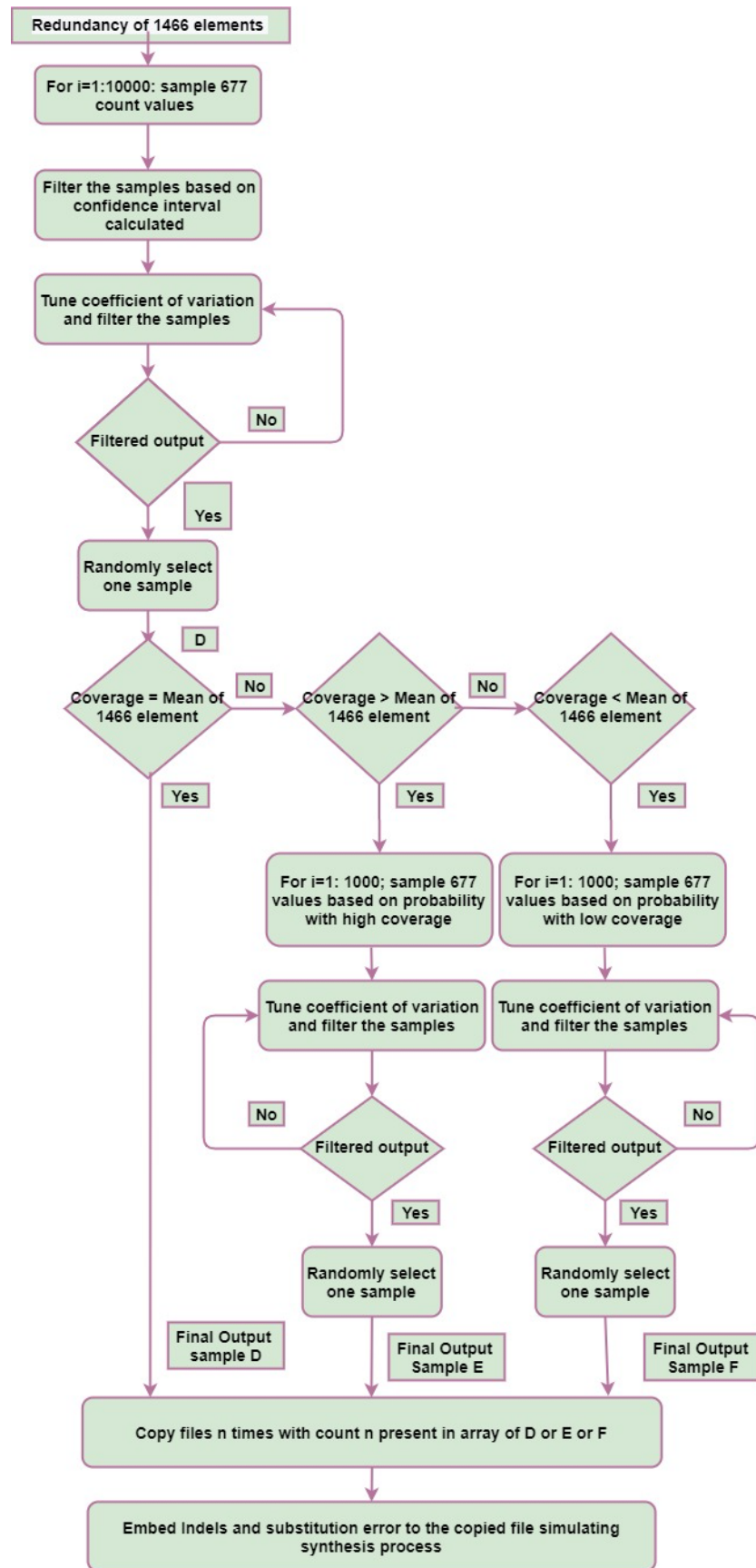


Figure 20: Flowchart for integrating sequencing coverage and synthesis error

Next, the tuned CV is determined. If the coverage value set from the user is the same as the actual mean (i.e mean of sample set A), the process is stopped. However, if the coverage value is greater or lesser than the original mean, the probability-based sampling is executed 1000 times and CV is tuned with values as shown in Table 6. Finally, we extracted one sample set after filtering.

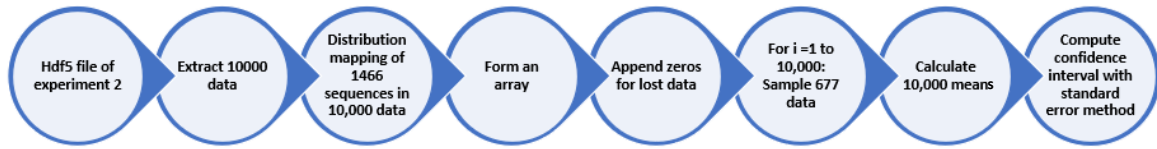


Figure 21: Computing Confidence Interval

Table 6: Confidence Interval and Coefficient of Variation for various distribution

Distribution	Range of mean through Confidence Interval	Coefficient of Variation (equal coverage)	Coefficient of Variation (higher coverage)	Coefficient of Variation (lower coverage)
Normal	[6.662, 6.982]	[1.61,1.63]	[1.61,1.63]	[1.10,1.77]
Log gamma	[13.354, 13.930]	[1.813, 1.815]	[1.750, 1.850]	[1.10,1.80]
Beta	[11.110, 11.695]	[1.49,1.50]	[1.49,1.50]	[1.10,1.50]

5.5 Integration of Preprocessing module in DeepSimulator

The Deep Simulator [37] has three sub modules such as sequence generator, signal generator and basecaller. Sequence generator is a sampling process performed in two ways. One way is splitting a read of larger length into ‘k’ reads with overlapped ends, and ‘k’ files are created consisting one of the ‘k’ reads. The other way is distributing each read present in a file (a file consisting of ‘k’ reads) into ‘k’ files with one read per file. The second way was chosen in designing the proposed DNA storage simulator. Those files are then sent to the signal generator, and one signal is obtained for each read which is further passed to basecaller to obtain base called sequence. For analysis purpose, the output of basecaller is passed to Minimap2 and original sequence is aligned with base called sequence to obtain the accuracy rate. Deep-simulator generates one output for each input sequence while with the proposed simulator, various copies of an input sequence are generated according to user-defined coverage and distribution. The output copies are embedded with errors randomly distributed simulating random errors existing in the practical DNA data storage system. As a result, at the receiver of a DNA data storage system, each original sequence is no longer with only one copy but several copies due to the redundant copies added in synthesizing, PCR amplification, and sequencing.

Through the analysis carried out with real-time data, after the sequence generation process, a module for preprocessing (Figure 20) is added in dotted line of Figure 22. In the proposed preprocessing module, three different distributions (normal, log gamma, beta) and customized coverage up to 62 are provided for selection. After that, the corresponding signal is generated simulating the operation of nanopore sequencer followed by a process of basecalling and alignment to compare the output with input sequence.

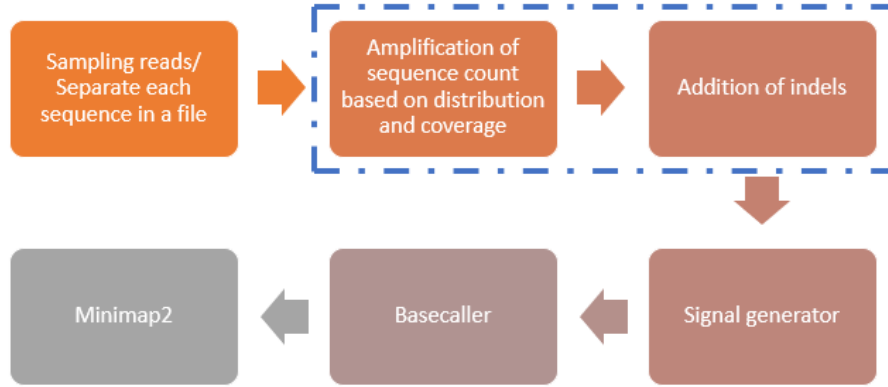


Figure 22: DNA Storage Channel Simulator- Dotted rectangle is integrated in existing DeepSimulator for simulating steps of synthesizing, PCR, sequencing certain amount of data (determined by specific coverage)

–Y is added as parameter for specifying coverage (varying from 1 to 62x) in dna_storage_channel_simulator.sh file, -d is added for specifying distribution (1-normal, 2-beta, 3-loggamma). -n -1 to avoid the splitting of reads. (e, f, g and o are the other parameters for specifying noise, frequency, text signal file and output aligned file respectively)

E.g.: / dna_storage_channel_simulator.sh -i input_677reads.txt -n -1 -d 1 -Y 60

5.6 Validation of the Proposed Simulator

DeepSimulator generates one output for each input sequence. Using proposed simulator, we get an output sequence with many copies as per user-defined coverage and distribution. Moreover, the errors should be randomly distributed in all copies to accurately design the DNA storage channel simulator since the copies of sequences readout have random errors in real-time.

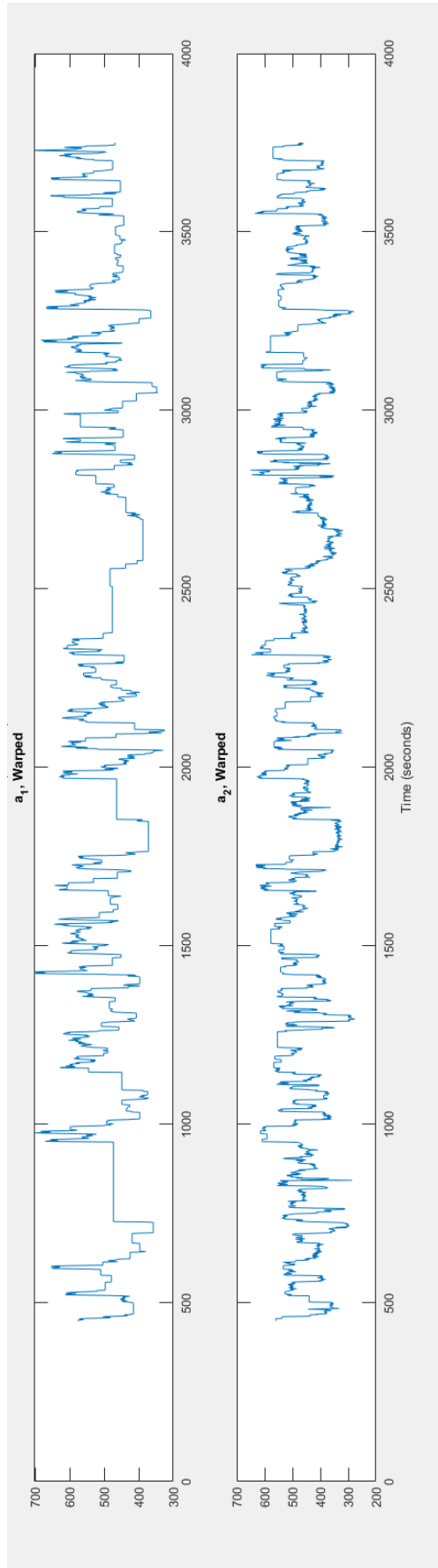
To validate the proposed simulator's accuracy, the outputs generated from the proposed simulator were compared to the outputs generated from the nanopore sequencer. Specifically, the signals obtained from the original nanopore were taken and compared with simulated signals after Dynamic Time Warping (DTW). The two signals (original and simulated signal) are stretched onto a standard set of instants such that the sum of Euclidean distance between the corresponding points is minimized. On doing that, the visual representation of the signal is similar as shown in Figures 23 and 24.

Table 7: Sample difference calculation between nanopore signal and simulated signals (having sample length of around 4000 due to biomolecular error after DTW) mapped to input encoded sequence of length 160

Sample difference between DeepSimulator and nanopore output	Sample difference between designed simulator and nanopore output
<u>86</u>	118, <u>27</u> , <u>27</u> , 145, 32, 86, 121, 118, 127, 33
12	67, 95, 10 , 12, 93, 99, 12, 101, 12, 101
17	6 , 27, 23, 23, 6 , 17, 47, 28, 16, 56
178	132, 179, 39, 21, 22, 147, 145, 48, 20 , 145

For validating the results of the proposed simulator, we have performed three analyses. Firstly, one Nanopore signal (in form of fast5) is compared with simulated signal after dynamic time warping. This applies to both simulated signals obtained from DeepSimulator and proposed simulator. With the addition of the synthesizing module in the designed simulator, the sequences are embedded with certain insertion, deletion, and substitution rate whereby the error rate at the receiver are closer to the real DNA data storage. We took four encoded input sequences from [31] and correspondingly four original output nanopore signals for validation. Each of the input sequences was processed in DeepSimulator, producing one simulated output per sequence. Following that, the sample difference between simulated and the original nanopore signal was calculated through correlation. Similarly, we have extracted 11 simulated copies per sequence using the proposed DNA storage channel simulator and computed the sample difference between the simulated and original nanopore signal as shown in Table 7. From the table, it is found that DeepSimulator generates a signal with a higher sample difference compared to the original output signal. In contrast, the proposed simulator produces certain output signals with lesser sample differences than the original output. The signals obtained from DeepSimulator have 16.7% to 88.7% higher sample differences than the signals generated by the proposed simulator. In Figure 26, the visual representation of signal is shown for signal having sample difference of 86 and 27 as underlined in Table 7. In this figure, the similarity of signal can be observed visually and also sample difference calculated out of 4000 length (time) is 86 and 27.

DNA Signals from Proposed Simulator and real nanopore Sequencer having difference of 27



DNA Signals from DeepSimulator and real nanopore Sequencer having difference of 86

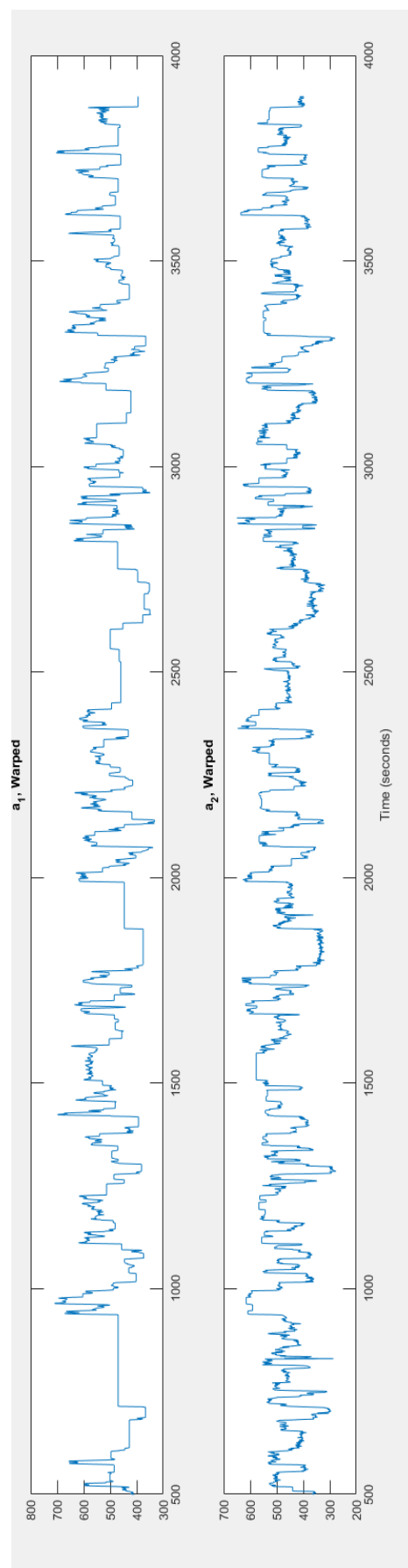


Figure 23: DWT warped signal (a₁ warped signal – simulated signal and a₂ warped signal – Real Nanopore signal)

Secondly, two redundant signals obtained from original nanopore is taken and compared with simulated signal after doing dynamic time warping. On doing that, the visual representation of signal looks similar. Here, we are comparing the simulated output with two redundant sets of real data and also finding out difference between two original nanopore. We observed that the real nanopore signal and its redundant real copy read by nanopore sequencer are having certain sample difference of 286 (as shown in Table 8) which can be mapped to same encoded input sequence. It further strengthens the proposed simulator's validation as signal copies generated from the proposed simulator also differ with significant sample difference. Now, using two real nanopore output signals of the same original sequence, validation is carried out by estimating sample difference between 11 simulated output and two copies. The differences between deep/designed simulators and the nanopore outputs (2 signals) were computed where we observed that the proposed simulator can produce outputs having 40 and 5 samples different from that of the original two nanopore signals while DeepSimulator's sample differences are 2681 and 5. To illustrate the similarity between the simulated signal and two raw nanopore signals, we display the third simulated copy that has the least overall sample difference from real nanopore signals (40 with respect one real copy and 48 with respect to the other as underlined in Table 8) after preforming DTW and cropping in Figure 24. Cross correlation graph in Figure 24 tells us maximum correlation observed between simulated and real nanopore signals. We also took another set of 2 samples and recorded the observations. From Table 9, it can be inferred that proposed simulator simulates the signal closer to real nanopore signal 2 having sample difference of 348 whereas the DeepSimulator simulates with difference of 1163. From Table 10, it can be inferred that proposed simulator simulates the signal closer to real nanopore signal where sample difference (654) is even lesser than sample difference obtained between two real nanopore signals 1 and 2 (857).

Table 8: Sample difference calculation between two nanopore signals and simulated signals (having sample length of around 4500 due to biomolecular error after DTW) mapped to input encoded sequence of length 160

Difference between two nanopore signals mapped to same encoded sequence: 286	Sample difference between DeepSimulator and nanopore output	Sample difference between designed simulator and nanopore output
Redundant Real Signal 1	2681	2680, <u>40</u> , 2694, 2681 50, 71, 40 , 2681, 2687, 2694
Redundant Real Signal 2	5	18, <u>48</u> , 64, 5 , 61 38, 47, 5 , 59, 40

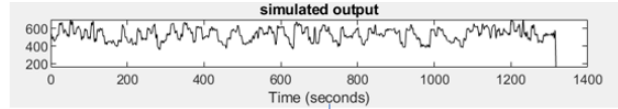
Table 9: Observation 1- Sample difference calculation of two nanopore signals of the same original sequence and simulated signals (having sample length of around 4000 due to biomolecular error after DTW) mapped to input encoded sequence of length 160

Difference between two nanopore signals mapped to same encoded sequence: 43	Sample Difference between DeepSimulator and Nanopore Output	Sample Difference between designed simulator and Nanopore Output
Redundant Real Signal 1	303	308,303,305,340,304,303,303, 272 ,342,279
Redundant Real Signal 2	1163	1163,1163,1163,1163, 348 ,1163,1163,353,1163,358

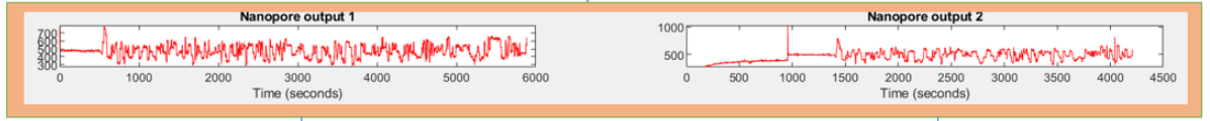
Table 10: Observation 2 - Sample difference calculation of two nanopore signals of the same original sequence and simulated signals (having sample length of around 3000 due to biomolecular error after DTW) mapped to input encoded sequence of length 160

Difference between two nanopore signals mapped to same encoded sequence: 857	Sample Difference between DeepSimulator and Nanopore Output	Sample Difference between designed simulator and Nanopore Output
Redundant Real Signal 1	657	677,657,673,670, 654 ,677,657,677,674,657
Redundant Real Signal 2	1362	1348 ,1362,1367,1372,1363,1365,1362,1365,1368,1357

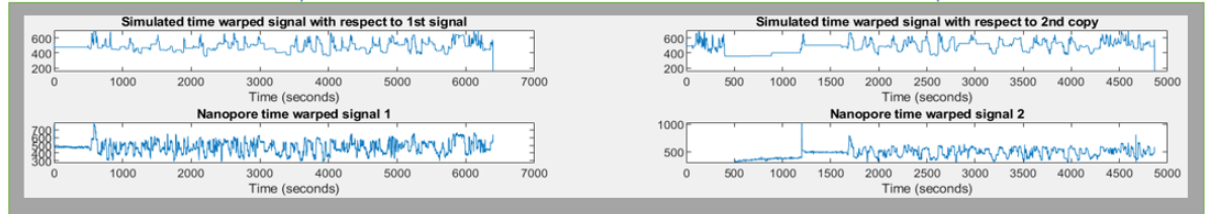
Simulated signal
mapped with
encoded sequence A



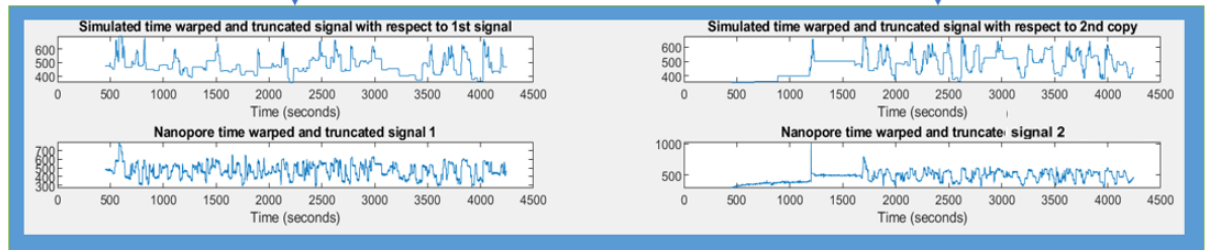
Two nanopore signal
mapped with
encoded sequence A



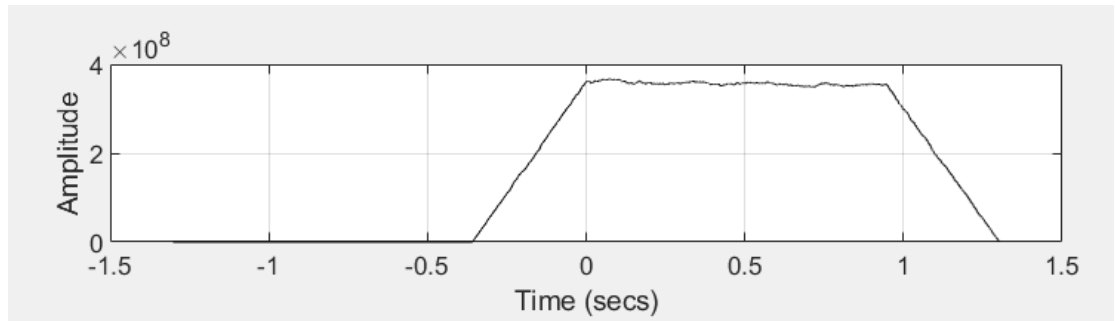
Time warped
simulated and
nanopore signal



Truncated simulated
and nanopore signal



Cross-correlation between Simulated signal and Nanopore signal 1



Cross-correlation between Simulated signal and Nanopore signal 2

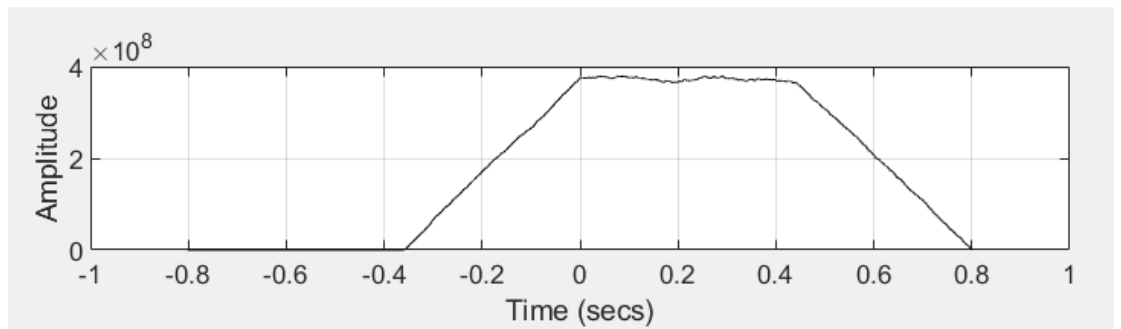


Figure 24: Analysis of signals: Simulated signals from the proposed simulator having the least sample difference (with respect to 2 real nanopore signals) are taken and represented by pictorial representation after performing DWT and cropping.

And third analysis is performed by comparing simulated sequence with real nanopore sequence with respect to biomolecular errors. To further validate, the biomolecular errors between the input encoded sequence and the nanopore sequence, input encoded sequence and simulated sequence from DeepSimulator, input encoded sequence and proposed simulated output are analyzed as shown in Figure 25. We observed that the proposed simulator produces an output having comparable biomolecular errors (Insertion: 5, Deletion: 2, Substitution: 4) to the nanopore sequenced output (Insertion: 5, Deletion: 2, Substitution: 5) (highlighted by box) whereas the DeepSimulator produces highly deviated error especially in deletion (Insertion: 3, Deletion: 11, Substitution: 6). Also, one important observation is that the primer sites are affected maximum with error compared to centre.

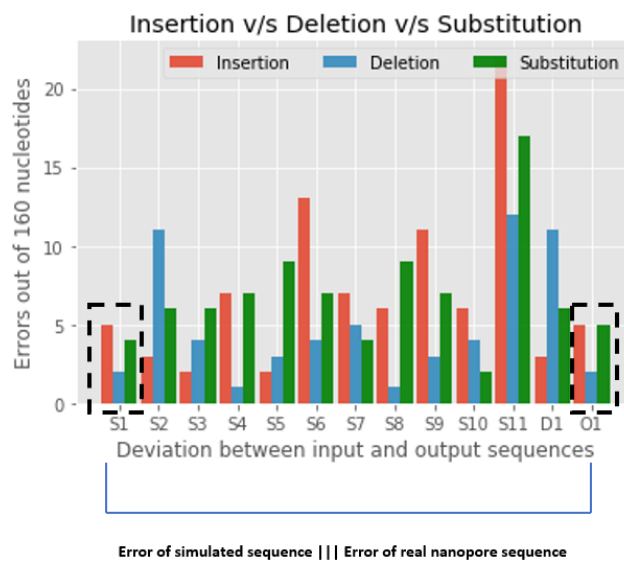


Figure 25: Biomolecular Errors in nanopore Sequencer (O1), proposed Simulator (S1-S11) and DeepSimulator (D1)

With Primer, the simulated output and original nanopore output is having higher difference in biomolecular errors such as insertion ranges between 12-18, deletion 2-5, and substitution 3-11. Without Primer, the simulated output and original nanopore output is having lower difference in biomolecular errors such as insertion ranges between 1-9, deletion between 1-4 and substitution 2-7. This is one of the major observations related to high error occurrence in primer binding site as shown in Figure 26. From the validation performed, it is observed that the proposed simulator better simulates the sequences/signal generated in a nanopore sequencing based on DNA data storage.

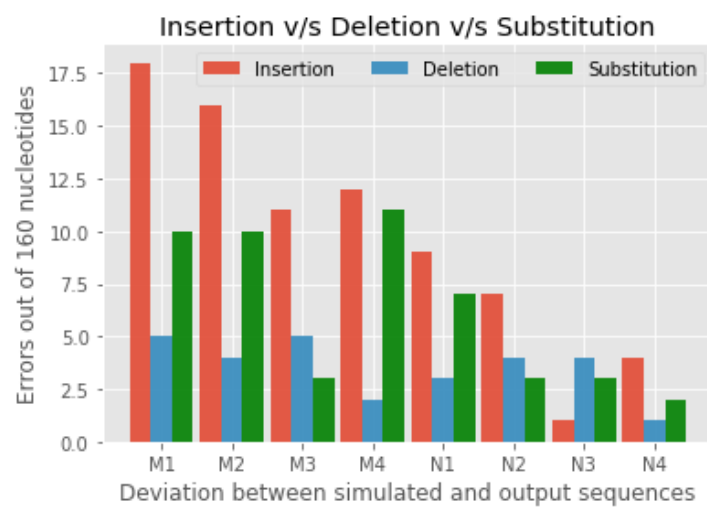


Figure 26: Biomolecular Errors with primer (M1-M4),
without primer (N1-N4)

5.7 Downstream processing of Simulated sequences

A study is conducted to understand the utilization of designed computational simulator by tuning its basecalling accuracy, internal parameters such as frequency and noise, coverage. This helps us to create the similar configuration in real time set up. We performed clustering of those simulated sequence using isONclust. Errors of entire elements in a cluster as well as errors in determined representative sequence is compared to determine the clustering algorithm performance along with other evaluation metric such as Homogeneity and Completeness.

isONclust is a clustering tool in which the sequenced read is passed and clustered. As a result, we get cluster representative in one file and cluster members in other file grouped through same number.

1. With the cluster member file from tool's output and mapping file from minimap2, each cluster belonging to original 677 sequence (we have used 677 number of sequences generated from our earlier encoding experiment) can be mapped and its member count can be determined. Through this process, an array can be formed with count of each sequence grouped as cluster.
2. The array formed after clustering process and array formed during preprocessing step in our proposed simulator can be evaluated using clustering metrics such as Homogeneity, and Completeness along with estimating the distribution. In addition, number of sequences that can be recovered is also found.
3. The cluster representative is compared with original 677 sequence and mapping file is created through minimap2
4. Using Sam tools, the indels and substitution error rate of all sequence and cluster representative sequence with respect to original 677 sequence is obtained. (Figure 27)

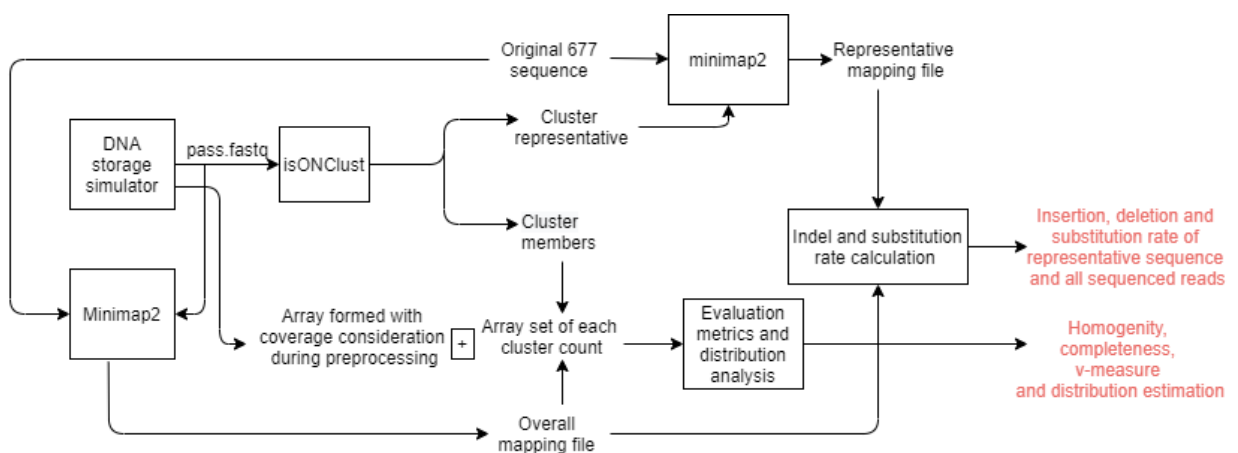


Figure 27: Flow diagram- Clustering, Clustering metric calculation and Evaluation of tool

When a coverage say x is specified, then in ideal case 677 sequence has to be sampled with mean of x redundancy. But in practical case, there is a sampling loss associated with 677 sequence itself. For

example, if we set higher coverage of 62 with log-gamma distribution and basecalling accuracy of 85.5 (set by tuning frequency and noise), 666 sequence is sampled with total count of 41974 ($677 * 62$) and 11 sequence is lost due to sampling.

The frequency and noise are tuned to different values and based on those parameter variation, basecalling accuracy is calculated by deep simulator internally. If basecalling accuracy is less, it produces less clusters on clustering through isONclust. To overcome this, in case if basecalling accuracy is low, the coverage can be increased thereby increasing number of clusters formed which results in increasing probability of recovering the sequence.

In initial experiments conducted, we observed and obtained certain values that lead to low accuracy. In addition, for those cases having low accuracy, the alignment check of extracted cluster representative with original data leads to very low similarity rate which means more coverage is required to achieve the maximum similarity rate.

The coverage is tuned to different values. For fixed basecalling accuracy, the similarity rate on comparison with original sequence is plotted. (Figure 28). For basecalling accuracy of above 86%, coverage of up to 30x is required and for basecalling accuracy above 80% and below 86%, similarity rate is very less even with higher coverage of 62x. We found the range of coverage required for different basecalling accuracy provided clustering tool is isONclust. Above 91% and from 86% to 90%, corresponding minimal coverage of 3x to 5x, 5x to 30x is required to achieve 100% recovery rate irrespective of mutation whereas for accuracy of 85%, 84%, 83%, 82%, 81%, 80% the coverage of 62x yields the recovery rate of 95%, 87%, 45%, 35%, 32%, 17%. In Figure 28, for basecalling accuracy above 90%, the coverage greater than 30x is not computed as it shows highest possibility to recovering even with lowest coverage value. For fixed coverage, basecalling accuracy is varied producing different number of clusters (Figure 29). These cluster numbers should be converged to original encoded sequence thereby reducing complexity to be dealt by decoder. In Figure 29, for high basecalling accuracy above 86%, coverage of 50x and 62x is not required to be computed owing to its high possibility to recover even with lowest coverage.

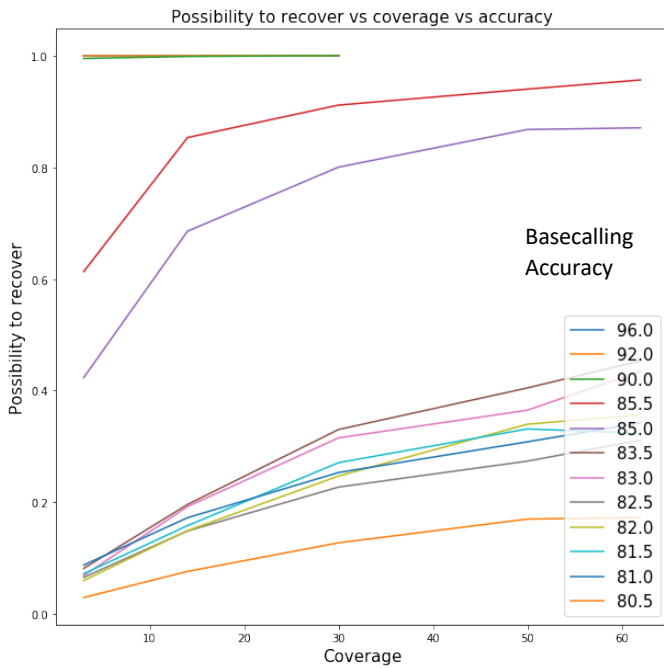


Figure 28: Possibility to recover data.

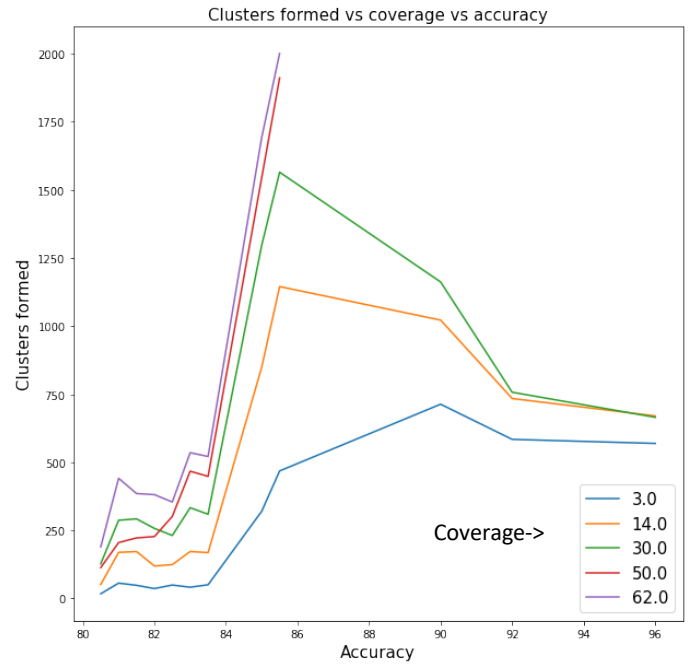


Figure 29: Formation of clusters with count

The clustering metrics are evaluated to determine the performance of clustering in terms of its accuracy (Figure 30 and Figure 31). Homogeneity means a cluster should contain only samples belonging to a single class. Completeness means assign all samples with the same true label to the same cluster. Both metrics are increasing while increasing the coverage size and basecalling accuracy.

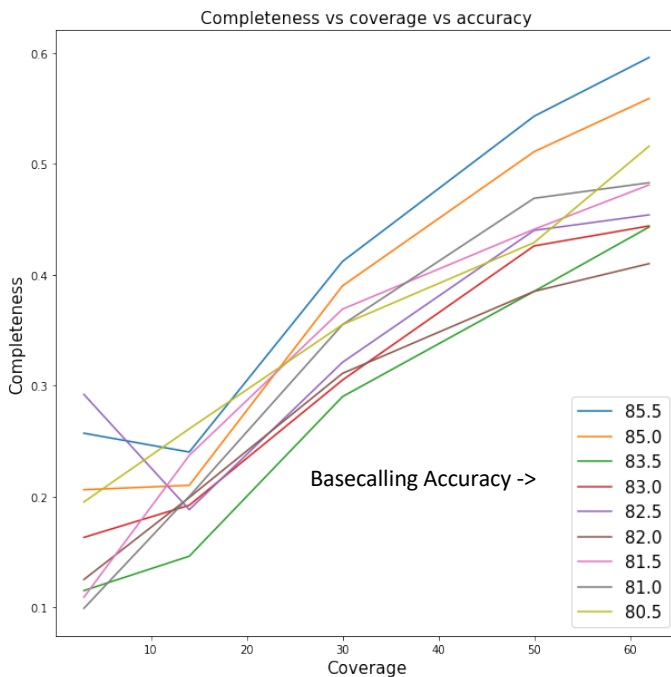


Figure 30: Completeness

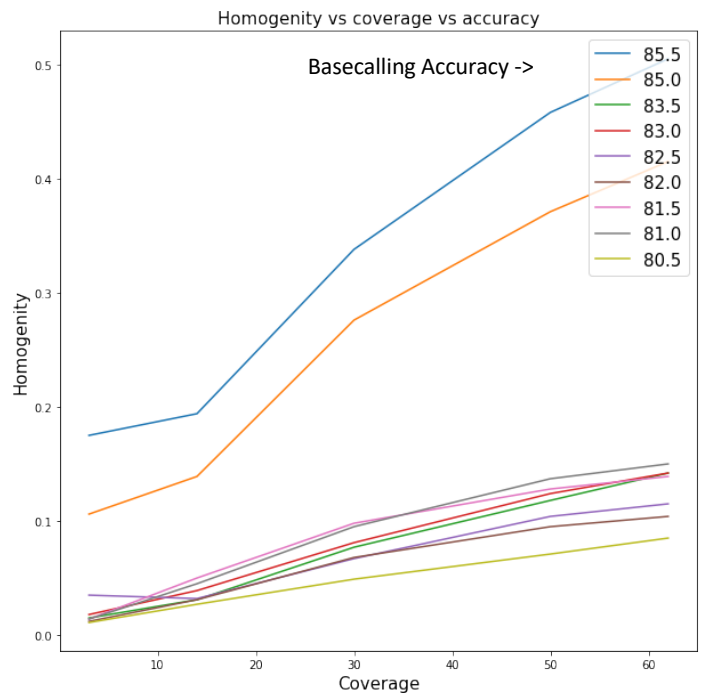


Figure 31: Homogeneity

With varying values of basecalling accuracy in x-axis and error rate in y-axis, error rate before clustering and after clustering is represented through graph (Figure 32). The graph implies, even at low accuracy, isONclust reduces the error at higher level with its detection of better representative sequence. This graph also implies that proposed simulator produces error rate upto 20% at less basecaller accuracy which is in concordance with real time experiment as in [31]

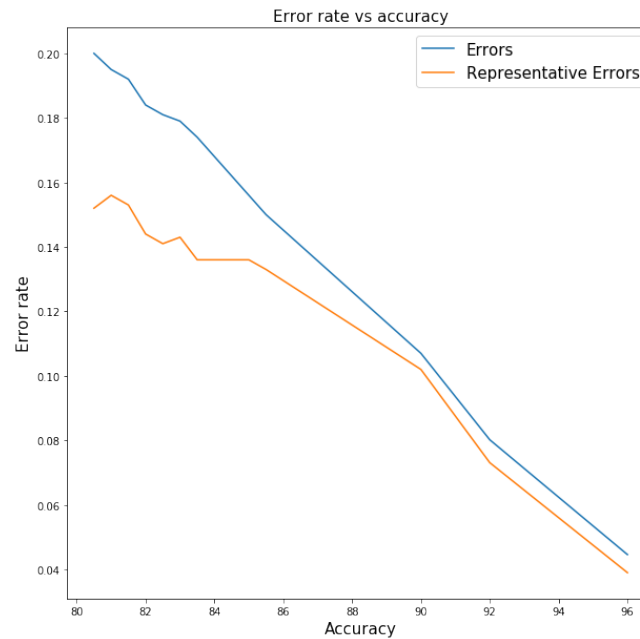


Figure 32: Error rate vs Accuracy

Indel and substitution rate of representative reads and sequenced reads for different basecalling accuracy is plotted (Figure 33). It is inferred from bar graph that the representative sequence of each clusters has less Insertion, deletion and substitution error compared to all sequence subsampled. In graph, DF1 represents Cluster elements and DF2 represents representative elements.

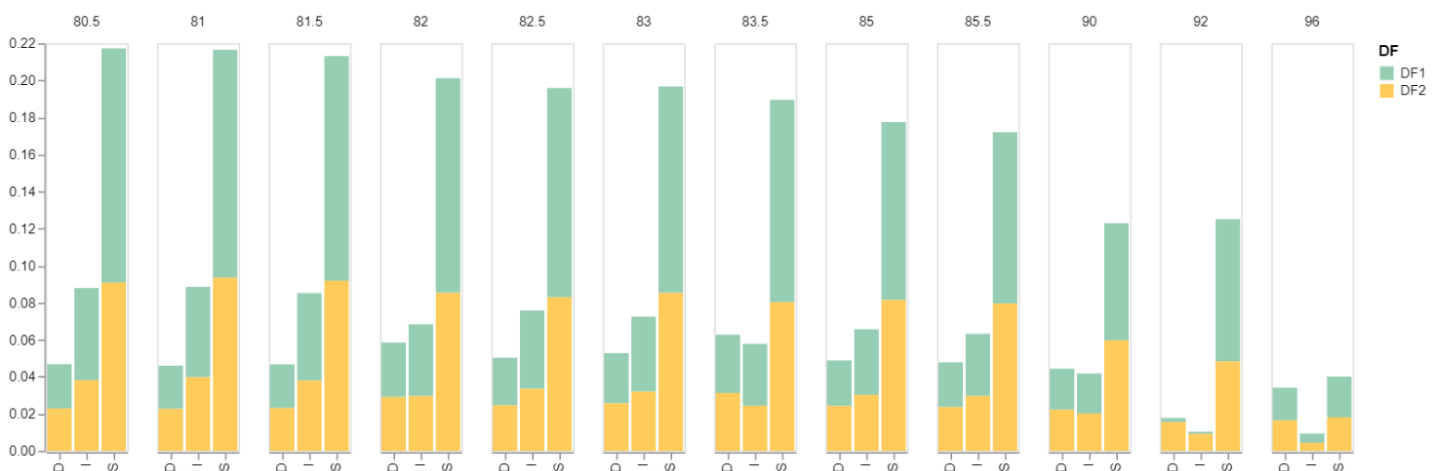


Figure 33: Comparison of varied error rate for all sequence and representative sequence

Length of sequence before clustering and after clustering is plotted (having basecalling 86% accuracy) (Figure 34). Sequence length of representative sequence is reduced when compared to all sequenced data distribution. In other words, there is a possibility of removal of insertion errors while detecting representative sequence.

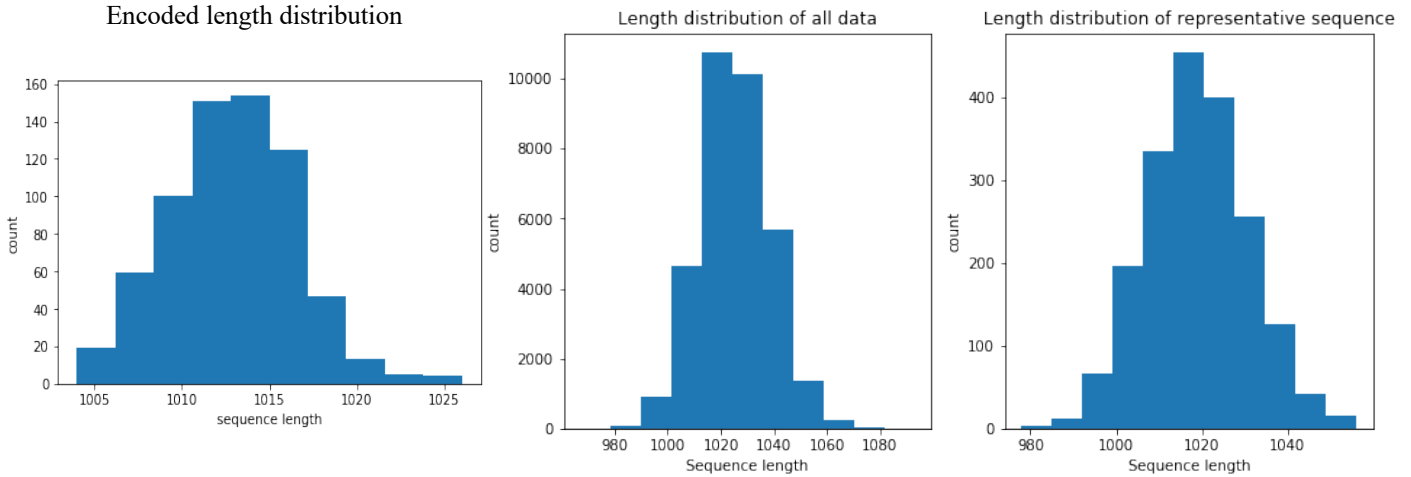


Figure 34: Sequence length distribution

5.8 Summary

In this work, we proposed a simulator depicting the nanopore sequencing-based DNA data storage channel, relieving the experiment burden for research in DNA data storage. An option to specify coverage for better adaptability to different sequencing redundancy with different experimental settings provided, which can be used to assess *logical and sequencing/physical redundancy*, thereby providing a reference for designing encoding/decoding schemes and reconstruction methods. The proposed simulator is a variant of DeepSimulator suited for the DNA data storage domain, preserving the same accuracy as its predecessor. Moreover, the proposed simulator is validated with sample difference and biomolecular errors calculation. Currently, the maximum coverage provided by the proposed simulator is 62x. And we also performed downstream processing with one clustering tool to explore the impact of estimating the representative sequence (mapped to input encoded sequence by reconstructing data back fed into simulator) with the highest similarity rate by varying the external parameters.

Chapter 6 Conclusion

In conclusion, different clustering tools and algorithm based on Illumina sequences for reconstructing encoded DNA sequences are studied, modified, and evaluated for minimal coverage and maximal recovery rate, which corresponds to low sequencing complexity and high recovery rate respectively. The decoding complexity after applying these mechanisms is also considered. Three datasets are extensively studied and experimented with different modified clustering tools and we observed that Starcode, Cooperative sequence clustering, and Majority nucleotide selection algorithm performs better for three datasets in terms of reducing coverage and still achieving maximum recovery rate. Majority nucleotide selection provides better performance in terms of reducing decoding complexity and cooperative is comparable to majority nucleotide selection algorithm whereas Starcode achieves the highest recovery rate with minimal coverage at the cost of increasing decoding complexity.

The benefit of portability in the Nanopore sequencer makes it more appropriate fit for DNA storage technology in recent days. For Nanopore sequencing, the basecalling accuracy is also one of the important parameters for finding out optimum coverage. Our design of computational simulator could help in analysing the basecalling accuracy in a simulated setup. The proposed computational simulator based on a nanopore sequencer depicting the entire DNA storage channel provides an option to specify coverage for increasing the redundancy and also synthesizing errors are integrated into the proposed model. Therefore, it can be used for assessing *logical and physical redundancy* thereby designing suitable encoding/decoding schemes and clustering techniques. In addition, the signals generated from the proposed simulator are validated by comparing them with real nanopore signals. And also, the utilization of this simulator is studied by clustering the simulated sequences and estimating the representatives for reconstructing encoded DNA sequences back and in addition, we also estimated coverage based on various base calling accuracy. The analysis with the clustering tool shows that the representative sequence extracted after clustering suffers less error rate compared to all other sequences.

6.1 Research Outcome

DNA storage technology, mutations, and clustering methodologies are discussed in chapter 2. Research questions and potential solutions are discussed in chapter 3. In chapter 4, a comparative analysis is performed on sequences obtained from Illumina sequencer with various clustering tools. In chapter 5, a DNA storage simulator model was proposed, which helps in deciding appropriate techniques at the coding level and downstream data processing level. The experiment conducted with real-time Illumina reads tells us the better clustering technique based on context such as sequencing complexity, decoding complexity, speed, and distribution after clustering. Because of its portability, there has been higher interest in using nanopore sequencers for performing sequencing action in DNA storage technology. Practically, to decide an encoding/decoding scheme and clustering methodology, a lot of trial-and-error

experiment has to be conducted. Instead, these experiments can be conducted in simulated environment using the computational simulator devised with coverage and distribution in the DNA storage domain and embedded synthesizing errors. We have validated the tool and have also shown that the proposed simulator is providing results with different accuracy levels assisting to decide a suitable clustering technique. In addition, a clustering experiment conducted with nanopore reads obtained from the simulator determines the range of coverage required for different basecalling accuracy.

6.2 Recommendations for future research

In Simulator model designed as well as different clustering techniques explored, there are still many questions remained to be solved. In future, research work will follow these aspects

- **Maximizing coverage size tuning limit of DNA storage channel simulator tool:** The coverage provided by the proposed simulator can be augmented in future works by using machine learning frameworks such as General Adversarial Networks.
- **Designing a new soft signal-based clustering method for Nanopore based storage:** Embedding soft clustering (signal based) with hard clustering (sequence based) may provide accurate result and also might aid in recovering the sequences with less error rate. Soft clustering can be performed on signal output as well as intermediate base caller output such as transition probability matrix using machine learning techniques such as spectral clustering, Agglomerative, K-means DTW, FFT based clustering etc.

PUBLICATIONS

- 1.) Subhasiny, Wang Yixin, Zhang Jiayu, Nur Sabrina, Erry Gunawan, Yong L.Guan, Noor-A-Rahim, Chueh L. Poh “Comparative Analysis of clustering methodologies in DNA Data Storage”, IEEE Symposiums on Computers and Communications, ISCC, 2022
[Status: Submitted for review]
- 2.) Subhasiny, Wang Yixin, Erry Gunawan, Yong L.Guan, Noor-A-Rahim, Chueh L. Poh “Channel simulator for Nanopore sequencing based DNA data storage, Bioinformatics,
[Status: Ongoing for submission]

REFERENCES

- [1] J. Shendure *et al.*, “DNA sequencing at 40: Past, present and future,” *Nature*, vol. 550, no. 7676, 2017, doi: 10.1038/nature24286.
- [2] A. Alharthi, V. Krotov, and M. Bowman, “Addressing barriers to big data,” *Bus. Horiz.*, vol. 60, no. 3, pp. 285–292, 2017, doi: 10.1016/j.bushor.2017.01.002.
- [3] A. O’Driscoll and R. D. Sleator, “Synthetic DNA: The next generation of big data storage,” *Bioengineered*, vol. 4, no. 3, pp. 3–6, 2013, doi: 10.4161/bioe.24296.
- [4] D. Limbachiya and M. K. Gupta, “Natural Data Storage: A Review on sending Information from now to then via Nature,” vol. V, no. 212, pp. 1–17, 2015, [Online]. Available: <http://arxiv.org/abs/1505.04890>.
- [5] Y. Dong, F. Sun, Z. Ping, Q. Ouyang, and L. Qian, “DNA storage: Research landscape and future prospects,” *Natl. Sci. Rev.*, vol. 7, no. 6, pp. 1092–1107, 2020, doi: 10.1093/nsr/nwaa007.
- [6] J. P. L. Cox, “Long-term data storage in DNA,” *Trends Biotechnol.*, vol. 19, no. 7, pp. 247–250, 2001, doi: 10.1016/S0167-7799(01)01671-7.
- [7] K. J. Tomek *et al.*, “Driving the scalability of DNA-based information storage systems order to reach practical capacities , new approaches for organizing and accessing information are needed . mimicking 5 TB of data and design and implement a nested file address system that i,” *bioRxiv*, 2019.
- [8] R. Heckel, I. Shomorony, K. Ramchandran, and D. N. C. Tse, “Fundamental limits of DNA storage systems,” *IEEE Int. Symp. Inf. Theory - Proc.*, pp. 3130–3134, 2017, doi: 10.1109/ISIT.2017.8007106.
- [9] K. Nakamura *et al.*, “Sequence-specific error profile of Illumina sequencers,” *Nucleic Acids Res.*, vol. 39, no. 13, 2011, doi: 10.1093/nar/gkr344.
- [10] R. M. Leggett and M. D. Clark, “A world of opportunities with nanopore sequencing,” *J. Exp. Bot.*, vol. 68, no. 20, pp. 5419–5429, 2017, doi: 10.1093/jxb/erx289.
- [11] S. Kosuri and G. M. Church, “Large-scale de novo DNA synthesis: Technologies and applications,” *Nat. Methods*, vol. 11, no. 5, pp. 499–507, 2014, doi: 10.1038/nmeth.2918.
- [12] N. Goldman *et al.*, “Towards practical, high-capacity, low-maintenance information storage in synthesized DNA,” *Nature*, vol. 494, no. 7435, pp. 77–80, 2013, doi: 10.1038/nature11875.
- [13] R. N. Grass, R. Heckel, M. Puddu, D. Paunescu, and W. J. Stark, “Robust chemical preservation

- of digital information on DNA in silica with error-correcting codes,” *Angew. Chemie - Int. Ed.*, vol. 54, no. 8, pp. 2552–2555, 2015, doi: 10.1002/anie.201411378.
- [14] S. M. H. T. Yazdi, H. M. Kiah, E. Garcia-Ruiz, J. Ma, H. Zhao, and O. Milenkovic, “DNA-Based Storage: Trends and Methods,” *IEEE Trans. Mol. Biol. Multi-Scale Commun.*, vol. 1, no. 3, pp. 230–248, 2015, doi: 10.1109/TMBMC.2016.2537305.
 - [15] S. M. H. Tabatabaei Yazdi, Y. Yuan, J. Ma, H. Zhao, and O. Milenkovic, “A Rewritable, Random-Access DNA-Based Storage System,” *Sci. Rep.*, vol. 5, pp. 1–10, 2015, doi: 10.1038/srep14138.
 - [16] J. Bornholt, R. Lopez, D. M. Carmean, L. Ceze, G. Seelig, and K. Strauss, “A DNA-based archival storage system,” *ACM SIGPLAN Not.*, vol. 51, no. 4, pp. 637–649, 2016, doi: 10.1145/2872362.2872397.
 - [17] Y. Erlich and D. Zielinski, “DNA Fountain enables a robust and efficient storage architecture,” *Science (80-.)*, vol. 355, no. 6328, pp. 950–954, 2017, doi: 10.1126/science.aaj2038.
 - [18] J. Jeong *et al.*, “Sequence analysis Cooperative Sequence Clustering and Decoding for DNA Storage System with Fountain Codes,” no. 1, 2021.
 - [19] L. Organick *et al.*, “Random access in large-scale DNA data storage,” *Nat. Biotechnol.*, vol. 36, no. 3, pp. 242–248, 2018, doi: 10.1038/nbt.4079.
 - [20] C. Rashtchian *et al.*, “Clustering billions of reads for DNA Data storage,” *Adv. Neural Inf. Process. Syst.*, vol. 2017-Decem, no. Nips, pp. 3361–3372, 2017.
 - [21] Y. Wang, M. Noor-A-Rahim, J. Zhang, E. Gunawan, Y. L. Guan, and C. L. Poh, “High capacity DNA data storage with variable-length Oligonucleotides using repeat accumulate code and hybrid mapping,” *J. Biol. Eng.*, vol. 13, no. 1, pp. 1–11, 2019, doi: 10.1186/s13036-019-0211-2.
 - [22] S. Chandak *et al.*, “Improved read/write cost tradeoff in DNA-based data storage using LDPC codes,” *2019 57th Annu. Allert. Conf. Commun. Control. Comput. Allert. 2019*, pp. 147–156, 2019, doi: 10.1109/ALLERTON.2019.8919890.
 - [23] M. Jain, H. E. Olsen, B. Paten, and M. Akeson, “The Oxford Nanopore MinION: delivery of nanopore sequencing to the genomics community,” *Genome Biol.*, vol. 17, no. 1, pp. 1–11, 2016, doi: 10.1186/s13059-016-1103-0.
 - [24] T. Laver *et al.*, “Assessing the performance of the Oxford Nanopore Technologies MinION,” *Biomol. Detect. Quantif.*, vol. 3, pp. 1–8, 2015, doi: 10.1016/j.bdq.2015.02.001.
 - [25] R. Lopez *et al.*, “DNA assembly for nanopore data storage readout,” *Nat. Commun.*, vol. 10, no.

- 1, pp. 1–9, 2019, doi: 10.1038/s41467-019-10978-4.
- [26] L. Ceze, J. Nivala, and K. Strauss, “Molecular digital data storage using DNA,” *Nat. Rev. Genet.*, vol. 20, no. 8, pp. 456–466, 2019, doi: 10.1038/s41576-019-0125-3.
 - [27] T. Lassmann and E. L. L. Sonnhammer, “Kalign - An accurate and fast multiple sequence alignment algorithm,” *BMC Bioinformatics*, vol. 6, pp. 1–9, 2005, doi: 10.1186/1471-2105-6-298.
 - [28] F. Sievers and D. G. Higgins, “Clustal Omega for making accurate alignments of many protein sequences,” *Protein Sci.*, vol. 27, no. 1, pp. 135–145, 2018, doi: 10.1002/pro.3290.
 - [29] R. C. Edgar, “MUSCLE: Multiple sequence alignment with high accuracy and high throughput,” *Nucleic Acids Res.*, vol. 32, no. 5, pp. 1792–1797, 2004, doi: 10.1093/nar/gkh340.
 - [30] S. M. Hossein TabatabaeiYazdi, R. Gabrys, and O. Milenkovic, “Portable and Error-Free DNA-Based Data Storage,” *Sci. Rep.*, vol. 7, no. 1, pp. 1–6, 2017, doi: 10.1038/s41598-017-05188-1.
 - [31] S. Chandak *et al.*, “Overcoming High Nanopore Basecaller Error Rates for DNA Storage via Basecaller-Decoder Integration and Convolutional Codes,” *ICASSP, IEEE Int. Conf. Acoust. Speech Signal Process. - Proc.*, vol. 2020-May, pp. 8822–8826, 2020, doi: 10.1109/ICASSP40776.2020.9053441.
 - [32] Y. J. Chen *et al.*, “Quantifying molecular bias in DNA data storage,” *Nat. Commun.*, vol. 11, no. 1, pp. 1–7, 2020, doi: 10.1038/s41467-020-16958-3.
 - [33] A. D. Tyler *et al.*, “Evaluation of Oxford Nanopore’s MinION Sequencing Device for Microbial Whole Genome Sequencing Applications,” *Sci. Rep.*, vol. 8, no. 1, pp. 1–12, 2018, doi: 10.1038/s41598-018-29334-5.
 - [34] R. Heckel, G. Mikutis, and R. N. Grass, “A Characterization of the DNA Data Storage Channel,” *Sci. Rep.*, vol. 9, no. 1, pp. 1–12, 2019, doi: 10.1038/s41598-019-45832-6.
 - [35] D. Aird *et al.*, “Characterizing and measuring bias in sequence data,” *Genome Biol.*, vol. 02, no. 5, p. 1, 2012, [Online]. Available: <http://genomebiology.com/2013/14/5/R51%5Cnhttp://www.biomedcentral.com/1471-2164/13/1%5Cnhttp://genomebiology.com/2011/12/2/R18>.
 - [36] Y. Li, R. Han, C. Bi, M. Li, S. Wang, and X. Gao, “DeepSimulator: A deep simulator for Nanopore sequencing,” *Bioinformatics*, vol. 34, no. 17, pp. 2899–2908, 2018, doi: 10.1093/bioinformatics/bty223.
 - [37] Y. Li *et al.*, “DeepSimulator1.5: A more powerful, quicker and lighter simulator for Nanopore sequencing,” *Bioinformatics*, vol. 36, no. 8, pp. 2578–2580, 2020, doi:

- 10.1093/bioinformatics/btz963.
- [38] K. Shimizu and K. Tsuda, “Slidesort: All pairs similarity search for short reads,” *Bioinformatics*, vol. 27, no. 4, pp. 464–470, 2011, doi: 10.1093/bioinformatics/btq677.
 - [39] E. Zorita, P. Cuscó, and G. J. Filion, “Starcode: Sequence clustering based on all-pairs search,” *Bioinformatics*, vol. 31, no. 12, pp. 1913–1919, 2015, doi: 10.1093/bioinformatics/btv053.
 - [40] B. T. James, B. B. Luczak, and H. Z. Girgis, “MeShClust: an intelligent tool for clustering DNA sequences,” *Nucleic Acids Res.*, vol. 46, no. 14, p. e83, 2018, doi: 10.1093/nar/gky315.
 - [41] L. Fu, B. Niu, Z. Zhu, S. Wu, and W. Li, “CD-HIT: Accelerated for clustering the next-generation sequencing data,” *Bioinformatics*, vol. 28, no. 23, pp. 3150–3152, 2012, doi: 10.1093/bioinformatics/bts565.
 - [42] M. Ghodsi, B. Liu, and M. Pop, “DNACLUST: Accurate and efficient clustering of phylogenetic marker genes,” *BMC Bioinformatics*, vol. 12, 2011, doi: 10.1186/1471-2105-12-271.
 - [43] J. Matar, H. EL Khoury, J. C. Charr, C. Guyeux, and S. Chrétien, “SpCLUST: Towards a fast and reliable clustering for potentially divergent biological sequences,” *Comput. Biol. Med.*, vol. 114, no. September, 2019, doi: 10.1016/j.compbiomed.2019.103439.
 - [44] J. Mei, J. Zhao, X. Yang, and W. Zhou, “Remote protein homology detection using a modularity-based approach,” *2011 Int. Conf. Inf. Sci. Technol. ICIST 2011*, pp. 1287–1291, 2011, doi: 10.1109/ICIST.2011.5765074.
 - [45] G. Palla, A. L. Barabási, and T. Vicsek, “Quantifying social group evolution,” *Nature*, vol. 446, no. 7136, pp. 664–667, 2007, doi: 10.1038/nature05670.
 - [46] V. D. Blondel, J. L. Guillaume, R. Lambiotte, and E. Lefebvre, “Fast unfolding of communities in large networks,” *J. Stat. Mech. Theory Exp.*, vol. 2008, no. 10, pp. 0–12, 2008, doi: 10.1088/1742-5468/2008/10/P10008.
 - [47] C. Marchet *et al.*, “De novo clustering of long reads by gene from transcriptomics data,” *Nucleic Acids Res.*, vol. 47, no. 1, pp. 1–13, 2019, doi: 10.1093/nar/gky834.
 - [48] H. Li, “Minimap2: Pairwise alignment for nucleotide sequences,” *Bioinformatics*, vol. 34, no. 18, pp. 3094–3100, 2018, doi: 10.1093/bioinformatics/bty191.
 - [49] H. Li and R. Durbin, “Fast and accurate long-read alignment with Burrows-Wheeler transform,” *Bioinformatics*, vol. 26, no. 5, pp. 589–595, 2010, doi: 10.1093/bioinformatics/btp698.
 - [50] C. Marchet *et al.*, “De novo clustering of long reads by gene from transcriptomics data,” *Nucleic Acids Res.*, vol. 47, no. 1, 2019, doi: 10.1093/nar/gky834.

- [51] M. Comin, A. Leoni, and M. Schimd, "Clustering of reads with alignment-free measures and quality values," *Algorithms Mol. Biol.*, vol. 10, no. 1, pp. 1–10, 2015, doi: 10.1186/s13015-014-0029-x.
- [52] K. Sahlin and P. Medvedev, "De Novo Clustering of Long-Read Transcriptome Data Using a Greedy, Quality Value-Based Algorithm," *J. Comput. Biol.*, vol. 27, no. 4, pp. 472–484, 2020, doi: 10.1089/cmb.2019.0299.
- [53] Wang, Yixin, et al. "Construction of bio-constrained code for DNA data storage." *IEEE Communications Letters* 23.6 (2019): 963-966.
- [54] R. Krishnakumar *et al.*, "Systematic and stochastic influences on the performance of the MinION nanopore sequencer across a range of nucleotide bias," *Sci. Rep.*, vol. 8, no. 1, pp. 1–13, 2018, doi: 10.1038/s41598-018-21484-w.
- [55] F. N. Muhamad, R. B. Ahmad, S. M. Asi, and M. N. Murad, "Performance Analysis of Needleman-Wunsch Algorithm (Global) and Smith-Waterman Algorithm (Local) in Reducing Search Space and Time for Dna Sequence Alignment," *J. Phys. Conf. Ser.*, vol. 1019, no. 1, 2018, doi: 10.1088/1742-6596/1019/1/012085.
- [56] M. Daszykowski and B. Walczak, "Density-Based Clustering Methods," *Compr. Chemom.*, vol. 2, pp. 635–654, 2009, doi: 10.1016/B978-044452701-1.00067-3.
- [57] Y. Cheng, "Mean Shift, Mode Seeking, and Clustering," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 17, no. 8, pp. 790–799, 1995, doi: 10.1109/34.400568.
- [58] R. Dong, L. He, R. L. He, and S. S. T. Yau, "A novel approach to clustering genome sequences using inter-nucleotide covariance," *Front. Pharmacol.*, vol. 10, no. FEB, 2019, doi: 10.3389/fgene.2019.00234.
- [59] A. J. C. Gower, G. J. S. Ross, S. Journal, R. Statistical, S. Series, and C. A. Statistics, "Minimum Spanning Trees and Single Linkage Cluster Analysis Published by: Wiley for the Royal Statistical Society Stable URL : <http://www.jstor.org/stable/2346439> Linked references are available on JSTOR for this article : Minimum Spanning Trees and Singl," vol. 18, no. 1, pp. 54–64, 2016.
- [60] Y. Wang, M. Noor-A-Rahim, J. Zhang, E. Gunawan, Y. L. Guan, and C. L. Poh, "Oligo Design with Single Primer Binding Site for High Capacity DNA-Based Data Storage," *IEEE/ACM Trans. Comput. Biol. Bioinforma.*, vol. 17, no. 6, pp. 2176–2182, 2020, doi: 10.1109/TCBB.2019.2940177.
- [61] E. A. G. Baker, S. Goodwin, W. R. McCombie, and O. Mendivil Ramos, "SiLiCO: A Simulator of Long Read Sequencing in PacBio and Oxford Nanopore," *bioRxiv*, p. 076901, 2016, doi:

10.1101/076901.

- [62] C. Yang, J. Chu, R. L. Warren, and I. Birol, “NanoSim: Nanopore sequence read simulator based on statistical characterization,” *Gigascience*, vol. 6, no. 4, pp. 1–6, 2017, doi: 10.1093/gigascience/gix010.
- [63] G. Trigeorgis, M. A. Nicolaou, B. W. Schuller, and S. Zafeiriou, “Deep Canonical Time Warping for Simultaneous Alignment and Representation Learning of Sequences,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 40, no. 5, pp. 1128–1138, 2018, doi: 10.1109/TPAMI.2017.2710047.
- [64] M. David, L. J. Dursi, D. Yao, P. C. Boutros, and J. T. Simpson, “Nanocall: An open source basecaller for Oxford Nanopore sequencing data,” *Bioinformatics*, vol. 33, no. 1, pp. 49–55, 2017, doi: 10.1093/bioinformatics/btw569.
- [65] V. Boža, P. Perešini, B. Brejová, and T. Vinař, “DeepNano-blitz: A fast base caller for MinION nanopore sequencers,” *Bioinformatics*, vol. 36, no. 14, pp. 4191–4192, 2020, doi: 10.1093/bioinformatics/btaa297.
- [66] F. J. Rang, W. P. Kloosterman, and J. de Ridder, “From squiggle to basepair: Computational approaches for improving nanopore sequencing read accuracy,” *Genome Biol.*, vol. 19, no. 1, pp. 1–11, 2018, doi: 10.1186/s13059-018-1462-9.
- [67] R. R. Wick, L. M. Judd, and K. E. Holt, “Performance of neural network basecalling tools for Oxford Nanopore sequencing,” *Genome Biol.*, vol. 20, no. 1, pp. 1–10, 2019, doi: 10.1186/s13059-019-1727-y.
- [68] H. Teng, M. D. Cao, M. B. Hall, T. Duarte, S. Wang, and L. J. M. Coin, “Chiron: translating nanopore raw signal directly into nucleotide sequence using deep learning,” *Gigascience*, vol. 7, no. 5, pp. 1–9, 2018, doi: 10.1093/gigascience/giy037.
- [69] Data age 2025: the digitization of the world from edge to core. <https://www.seagate.com/files/www-content/our-story/trends/files/idc-seagate-dataage-whitepaper.pdf>
- [70] Flappie: <https://github.com/nanoporetech/flappie>, Last accessed: October 3, 2019
- [71] Scrappie: <https://github.com/nanoporetech/scrappie>